

EULER EQUATION SOLVER

Finite Volume Method for Hyperbolic Equations (AS5460)

Jul-Nov 2026*

Kanak Agarwal[1]

[1] Department of Aerospace Engineering, Indian Institute of Technology, Madras, Chennai, India - 600036

* Submitted on November 11, 2026

Contents

1 Introduction **1**

2 Numerical Flux Schemes **1**

2.1 Steger-Warming Flux Vector Splitting (SW-FVS) 1

2.2 Van Leer Flux Vector Splitting (VL-FVS) 2

2.3 AUSM Flux Splitting Scheme 3

2.4 Roe's Approximate Riemann Solver (Roe-FDS) 4

3 Time Integration Schemes **5**

3.1 Forward Euler Method 5

3.2 Classical Fourth-Order Runge-Kutta (RK4) Method 6

3.3 CFL-Based Time-Step Calculation 6

4 Initialisation and Simulation Details **7**

4.1 Initial Condition 1 7

4.2 Initial Condition 2 7

4.3 Initial Condition 3 8

4.4 Boundary Conditions 8

4.5 Numerical Implementation 8

5 Exact Riemann Solution **9**

5.1 Pressure Function 9

5.2 Star Region Properties 10

5.3 Wave Speeds 11

5.4 Solution Inside the Rarefaction Fans 11

5.5 Evaluation of the Exact Solution 12

5.6 Implementation 13

6 Results and Discussion **13**

6.1 Initial Condition 1: Stationary Contact Discontinuity 13

6.1.1 Steger-Warming Flux Scheme 14

6.1.1.1 Forward Euler Time Integration 14

6.1.1.2 Runge-Kutta 4 Time Integration 15

6.1.2 Van Leer (Local) Flux Scheme 16

6.1.2.1 Forward Euler Time Integration 16

6.1.2.2 Runge-Kutta 4 Time Integration 17

6.1.3 Van Leer (Common) Flux Scheme 18

6.1.3.1 Forward Euler Time Integration 18

6.1.3.2 Runge-Kutta 4 Time Integration 19

6.1.4 AUSM Flux Scheme 20

6.1.4.1 Forward Euler Time Integration 20

6.1.4.2 Runge-Kutta 4 Time Integration 21

6.1.5 Roe Flux Scheme 22

6.1.5.1 Forward Euler Time Integration 22

6.1.5.2 Runge-Kutta 4 Time Integration . . . 23
6.2 Initial Condition 2: Stationary Normal Shock . . . 24
6.2.1 Steger–Warming Flux Scheme . . . 24
6.2.1.1 Forward Euler Time Integration . . . 24
6.2.1.2 Runge-Kutta 4 Time Integration . . . 25
6.2.2 Van Leer (Local) Flux Scheme . . . 26
6.2.2.1 Forward Euler Time Integration . . . 26
6.2.2.2 Runge-Kutta 4 Time Integration . . . 27
6.2.3 Van Leer (Common) Flux Scheme . . . 27
6.2.4 AUSM Flux Scheme . . . 28
6.2.4.1 Forward Euler Time Integration . . . 28
6.2.4.2 Runge-Kutta 4 Time Integration . . . 29
6.2.5 Roe Flux Scheme . . . 30
6.2.5.1 Forward Euler Time Integration . . . 30
6.2.5.2 Runge-Kutta 4 Time Integration . . . 31
6.3 Initial Condition 3: Rarefaction-Contact-Shock Interaction . . . 32
6.3.1 Steger-Warming Flux Scheme . . . 32
6.3.1.1 Forward Euler Time Integration . . . 32
6.3.1.2 Runge-Kutta 4 Time Integration . . . 33
6.3.2 Van Leer (Local) Flux Scheme . . . 34
6.3.2.1 Forward Euler Time Integration . . . 34
6.3.2.2 Runge-Kutta 4 Time Integration . . . 35
6.3.3 Van Leer (Common) Flux Scheme . . . 36
6.3.3.1 Forward Euler Time Integration . . . 36
6.3.3.2 Runge-Kutta 4 Time Integration . . . 37
6.3.4 AUSM Flux Scheme . . . 38
6.3.4.1 Forward Euler Time Integration . . . 38
6.3.4.2 Runge-Kutta 4 Time Integration . . . 39
6.3.5 Roe Flux Scheme . . . 40
6.3.5.1 Forward Euler Time Integration . . . 40
6.3.5.2 Runge-Kutta 4 Time Integration . . . 41
7 Conclusion **42**
8 References **42**
A FORTRAN Implementation **42**
A.1 Main Program . . . 42
A.2 Variable Declaration Module . . . 45
A.3 Exact Solution Module . . . 47
A.4 Subroutine Module . . . 50
B Python Post-Processing **62**

Figures

1 Comparison of numerical and exact solutions for Initial Condition 1 using the Steger-Warming flux scheme with Forward Euler time integration at $t = 0.0020$. 14
2 Comparison of numerical and exact solutions for Initial Condition 1 using the Steger-Warming flux scheme with classical fourth-order Runge-Kutta (RK4) time integration at $t = 0.0020$. 15
3 Comparison of numerical and exact solutions for Initial Condition 1 using the Van Leer (Local) flux scheme with Forward Euler time integration at $t = 0.0020$. 16
4 Comparison of numerical and exact solutions for Initial Condition 1 using the Van Leer (Local) flux scheme with RK4 time integration at $t = 0.0020$. 17
5 Comparison of numerical and exact solutions for Initial Condition 1 using the Van Leer (Common) flux scheme with Forward Euler time integration at $t = 0.0020$. 18
6 Comparison of numerical and exact solutions for Initial Condition 1 using the Van Leer (Common) flux scheme with RK4 time integration at $t = 0.0020$. 19
7 Comparison of numerical and exact solutions for Initial Condition 1 using the AUSM flux scheme with Forward Euler time integration at $t = 0.0020$. 20
8 Comparison of numerical and exact solutions for Initial Condition 1 using the AUSM flux scheme with RK4 time integration at $t = 0.0020$. 21
9 Comparison of numerical and exact solutions for Initial Condition 1 using the Roe flux scheme with Forward Euler time integration at $t = 0.0020$. 22
10 Comparison of numerical and exact solutions for Initial Condition 1 using the Roe flux scheme with RK4 time integration at $t = 0.0020$. 23
11 Comparison of numerical and exact solutions for Initial Condition 2 using the Steger–Warming flux scheme with Forward Euler time integration at $t = 0.0020$. 24
12 Comparison of numerical and exact solutions for Initial Condition 2 using the Steger–Warming flux scheme with RK4 time integration at $t = 0.0020$. 25
13 Comparison of numerical and exact solutions for Initial Condition 2 using the Van Leer (Local) flux scheme with Forward Euler time integration at $t = 0.0020$. 26
14 Comparison of numerical and exact solutions for Initial Condition 2 using the Van Leer (Local) flux scheme with RK4 time integration at $t = 0.0020$. 27
15 Comparison of numerical and exact solutions for Initial Condition 2 using the AUSM flux scheme with Forward Euler time integration at $t = 0.0020$. 28
16 Comparison of numerical and exact solutions for Initial Condition 2 using the AUSM flux scheme with RK4 time integration at $t = 0.0020$. 29
17 Comparison of numerical and exact solutions for Initial Condition 2 using the Roe flux scheme with Forward Euler time integration at $t = 0.0020$. 30
18 Comparison of numerical and exact solutions for Initial Condition 2 using the Roe flux scheme with RK4 time integration at $t = 0.0020$. 31
19 Comparison of numerical and exact solutions for Initial Condition 3 using the Steger–Warming flux scheme with Forward Euler time integration at $t = 0.0020$. 32

20 Comparison of numerical and exact solutions for Initial Condition 3 using the Steger–Warming flux scheme with RK4 time integration at $t = 0.0020$. 33
21 Comparison of numerical and exact solutions for Initial Condition 3 using the Van Leer (Local) flux scheme with Forward Euler time integration at $t = 0.0020$. 34
22 Comparison of numerical and exact solutions for Initial Condition 3 using the Van Leer (Local) flux scheme with RK4 time integration at $t = 0.0020$. 35
23 Comparison of numerical and exact solutions for Initial Condition 3 using the Van Leer (Common) flux scheme with Forward Euler time integration at $t = 0.0020$. 36
24 Comparison of numerical and exact solutions for Initial Condition 3 using the Van Leer (Common) flux scheme with RK4 time integration at $t = 0.0020$. 37
25 Comparison of numerical and exact solutions for Initial Condition 3 using the AUSM flux scheme with Forward Euler time integration at $t = 0.0020$. 38
26 Comparison of numerical and exact solutions for Initial Condition 3 using the AUSM flux scheme with RK4 time integration at $t = 0.0020$. 39
27 Comparison of numerical and exact solutions for Initial Condition 3 using the Roe flux scheme with Forward Euler time integration at $t = 0.0020$. 40
28 Comparison of numerical and exact solutions for Initial Condition 3 using the Roe flux scheme with RK4 time integration at $t = 0.0020$. 41

1 Introduction

This report presents the results of the inviscid Euler equation solver developed as a part of the AS5460 course at the Indian Institute of Technology, Madras.

2 Numerical Flux Schemes

The one-dimensional compressible Euler equations in conservative form are given by,

$$\frac{\partial \mathbf{U}}{\partial t} + \frac{\partial \mathbf{F}}{\partial x} = 0 \tag{1}$$

where the conservative variable vector and physical flux vector are defined as,

$$\mathbf{U} = \begin{Bmatrix} \rho \\ \rho u \\ \rho E \end{Bmatrix}, \qquad \mathbf{F} = \begin{Bmatrix} \rho u \\ \rho u^2 + p \\ u(\rho E + p) \end{Bmatrix} \tag{2}$$

The total specific energy is given by,

$$E = \frac{p}{(\gamma - 1)\rho} + \frac{u^2}{2} \tag{3}$$

where ρ, u, p, and γ denote density, velocity, pressure, and the ratio of specific heats, respectively. The finite-volume discretisation is written as,

$$\frac{d\mathbf{U}_i}{dt} = -\frac{1}{\Delta x}\left(\mathbf{F}_{i+\frac{1}{2}} - \mathbf{F}_{i-\frac{1}{2}}\right) \tag{4}$$

where $\mathbf{F}_{i+\frac{1}{2}}$ represents the numerical flux at the interface between cells i and $i+1$.

2.1 Steger-Warming Flux Vector Splitting (SW-FVS)

The Steger-Warming scheme employs characteristic decomposition to split the physical flux into positive and negative contributions,

$$\mathbf{F}(\mathbf{U}) = \mathbf{F}^{+}(\mathbf{U}) + \mathbf{F}^{-}(\mathbf{U}) \tag{5}$$

Using the eigenvalue decomposition of the flux Jacobian,

$$\mathbf{A} = \mathbf{R}\mathbf{\Lambda}\mathbf{R}^{-1} \tag{6}$$

the positive and negative flux components are defined as,

$$\mathbf{F}^{\pm} = \mathbf{R}\mathbf{\Lambda}^{\pm}\mathbf{R}^{-1}\mathbf{U} \tag{7}$$

where,

$$\mathbf{\Lambda}^{\pm} = \mathrm{diag}\left(\lambda_1^{\pm}, \lambda_2^{\pm}, \lambda_3^{\pm}\right) \tag{8}$$

and the split eigenvalues are,

$$\lambda_k^{\pm} = \frac{1}{2}\left(\lambda_k \pm |\lambda_k|\right) \tag{9}$$

For the one-dimensional Euler equations, the eigenvalues are,

$$\lambda_1 = u, \qquad \lambda_2 = u + a, \qquad \lambda_3 = u - a \tag{10}$$

where $a = \sqrt{\gamma p/\rho}$ is the local speed of sound. The numerical flux at the interface is therefore,

$$\boxed{\mathbf{F}^{\mathrm{SW}}_{i+\frac{1}{2}} = \mathbf{F}^{+}(\mathbf{U}_L) + \mathbf{F}^{-}(\mathbf{U}_R)} \tag{11}$$

where the subscripts L and R denote the left and right states, respectively.

2.2 Van Leer Flux Vector Splitting (VL-FVS)

The Van Leer scheme splits the flux based on the local Mach number,

$$M = \frac{u}{a} \tag{12}$$

The flux is decomposed into positive and negative components,

$$\mathbf{F} = \mathbf{F}^{+} + \mathbf{F}^{-} \tag{13}$$

For the positive flux contribution,

$$\mathbf{F}^{+} = \begin{cases} \mathbf{F}, & M \geq 1, \\ \mathbf{0}, & M \leq -1, \\ \mathbf{F}^{+}_{\mathrm{sub}}, & |M| < 1, \end{cases} \tag{14}$$

where,

$$\mathbf{F}^{+}_{\mathrm{sub}} = \frac{\rho a}{4}(M+1)^2 \begin{Bmatrix} 1 \\ \dfrac{2a}{\gamma}\left(M + \dfrac{\gamma-1}{2}\right) \\ \dfrac{2a^2}{\gamma^2-1}\left[\dfrac{\gamma-1}{2}M^2 + M + \dfrac{1}{\gamma+1}\right] \end{Bmatrix} \tag{15}$$

Similarly, the negative flux contribution is,

$$\mathbf{F}^{-} = \begin{cases} \mathbf{F}, & M \leq -1, \\ \mathbf{0}, & M \geq 1, \\ \mathbf{F}^{-}_{\mathrm{sub}}, & |M| < 1, \end{cases} \tag{16}$$

with,

$$\mathbf{F}^{-}_{\text{sub}} = -\frac{\rho a}{4}(M-1)^2 \begin{Bmatrix} 1 \\ \frac{2a}{\gamma}\left(M - \frac{\gamma-1}{2}\right) \\ \frac{2a^2}{\gamma^2-1}\left[\frac{\gamma-1}{2}M^2 - M + \frac{1}{\gamma+1}\right] \end{Bmatrix} \tag{17}$$

The interface flux is obtained by combining the positive contribution from the left state and the negative contribution from the right state.

$$\boxed{\mathbf{F}^{\text{VL}}_{i+\frac{1}{2}} = \mathbf{F}^{+}(\mathbf{U}_L) + \mathbf{F}^{-}(\mathbf{U}_R)} \tag{18}$$

Two variants are considered in the implementation.

- Local sound speed - The left and right states use their respective sound speeds, a_L and a_R.
- Common sound speed - Both states use $a_{\text{common}} = \max(a_L, a_R)$.

2.3 AUSM Flux Splitting Scheme

The Advection Upstream Splitting Method (AUSM) separates the convective and pressure contributions to the numerical flux using Mach-number and pressure splitting functions. The interface Mach number is calculated as,

$$M_{i+\frac{1}{2}} = \mathcal{M}^{+}(M_L) + \mathcal{M}^{-}(M_R) \tag{19}$$

where the Mach splitting functions are,

$$\mathcal{M}^{+}(M) = \begin{cases} \frac{1}{2}(M + |M|), & |M| \geq 1, \\ \frac{1}{4}(M+1)^2, & |M| < 1, \end{cases} \tag{20}$$

and,

$$\mathcal{M}^{-}(M) = \begin{cases} \frac{1}{2}(M - |M|), & |M| \geq 1, \\ -\frac{1}{4}(M-1)^2, & |M| < 1 \end{cases} \tag{21}$$

The interface pressure is obtained using the pressure splitting functions.

$$p_{i+\frac{1}{2}} = \mathcal{P}^{+}(M_L)p_L + \mathcal{P}^{-}(M_R)p_R \tag{22}$$

where,

$$\mathcal{P}^{+}(M) = \begin{cases} \frac{1}{2}(1 + \text{sgn}(M)), & |M| \geq 1, \\ \frac{1}{4}(M+1)^2(2-M), & |M| < 1 \end{cases} \tag{23}$$

and,

$$\mathcal{P}^{-}(M) = \begin{cases} \dfrac{1}{2}(1 - \mathrm{sgn}(M)), & |M| \geq 1, \\ \dfrac{1}{4}(M-1)^2(2+M), & |M| < 1 \end{cases} \tag{24}$$

The reference Mach numbers are calculated using a common interface sound speed.

$$a_{i+\frac{1}{2}} = \max(a_L, a_R), \qquad M_L = \frac{u_L}{a_{i+\frac{1}{2}}}, \qquad M_R = \frac{u_R}{a_{i+\frac{1}{2}}} \tag{25}$$

The interface mass flux is then given by,

$$\dot{m}_{i+\frac{1}{2}} = \begin{cases} a_{i+\frac{1}{2}} M_{i+\frac{1}{2}} \rho_L, & M_{i+\frac{1}{2}} \geq 0, \\ a_{i+\frac{1}{2}} M_{i+\frac{1}{2}} \rho_R, & M_{i+\frac{1}{2}} < 0 \end{cases} \tag{26}$$

The upwind velocity and total enthalpy are selected from the corresponding upstream state. The numerical flux is,

$$\boxed{\mathbf{F}^{\mathrm{AUSM}}_{i+\frac{1}{2}} = \begin{Bmatrix} \dot{m}_{i+\frac{1}{2}} \\ \dot{m}_{i+\frac{1}{2}} u_{\mathrm{up}} + p_{i+\frac{1}{2}} \\ \dot{m}_{i+\frac{1}{2}} H_{\mathrm{up}} \end{Bmatrix}} \tag{27}$$

where,

$$H = E + \frac{p}{\rho} \tag{28}$$

is the total specific enthalpy.

2.4 Roe's Approximate Riemann Solver (Roe-FDS)

Roe's scheme constructs a linearised approximation to the Riemann problem using Roe-averaged flow properties. The Roe-averaged velocity and total enthalpy are defined as,

$$\widetilde{u} = \frac{\sqrt{\rho_L} u_L + \sqrt{\rho_R} u_R}{\sqrt{\rho_L} + \sqrt{\rho_R}}, \qquad \widetilde{H} = \frac{\sqrt{\rho_L} H_L + \sqrt{\rho_R} H_R}{\sqrt{\rho_L} + \sqrt{\rho_R}} \tag{29}$$

The Roe-averaged speed of sound is,

$$\widetilde{a} = \sqrt{(\gamma - 1)\left(\widetilde{H} - \frac{\widetilde{u}^2}{2}\right)} \tag{30}$$

The eigenvalues of the Roe-averaged flux Jacobian are,

$$\widetilde{\lambda}_1 = \widetilde{u}, \qquad \widetilde{\lambda}_2 = \widetilde{u} + \widetilde{a}, \qquad \widetilde{\lambda}_3 = \widetilde{u} - \widetilde{a} \tag{31}$$

The corresponding right eigenvectors are

$$\widetilde{\mathbf{R}} = \begin{bmatrix} 1 & 1 & 1 \\ \widetilde{u} & \widetilde{u}+\widetilde{a} & \widetilde{u}-\widetilde{a} \\ \dfrac{\widetilde{u}^2}{2} & \widetilde{H}+\widetilde{u}\widetilde{a} & \widetilde{H}-\widetilde{u}\widetilde{a} \end{bmatrix} \tag{32}$$

The wave strengths are calculated as

$$\alpha_1 = \Delta\rho - \frac{\Delta p}{\widetilde{a}^2}, \qquad \alpha_2 = \frac{\Delta p + \widetilde{\rho}\widetilde{a}\Delta u}{2\widetilde{a}^2}, \qquad \alpha_3 = \frac{\Delta p - \widetilde{\rho}\widetilde{a}\Delta u}{2\widetilde{a}^2} \tag{33}$$

where

$$\widetilde{\rho} = \sqrt{\rho_L \rho_R} \tag{34}$$

The Roe numerical flux is then given by,

$$\boxed{\mathbf{F}^{\text{Roe}}_{i+\frac{1}{2}} = \frac{1}{2}\left[\mathbf{F}(\mathbf{U}_L) + \mathbf{F}(\mathbf{U}_R)\right] - \frac{1}{2}\sum_{k=1}^{3} |\widetilde{\lambda}_k| \alpha_k \widetilde{\mathbf{r}}_k} \tag{35}$$

where $\widetilde{\mathbf{r}}_k$ denotes the k-th Roe-averaged right eigenvector.

3 Time Integration Schemes

The semi-discrete finite-volume equations are integrated in time using two explicit time-stepping schemes, Forward Euler and the classical fourth-order Runge-Kutta (RK4) method. Defining the spatial residual as,

$$\mathcal{L}(\mathbf{U}) = -\frac{1}{\Delta x}\left(\mathbf{F}_{i+\frac{1}{2}} - \mathbf{F}_{i-\frac{1}{2}}\right) \tag{36}$$

the governing equations can be expressed as,

$$\frac{d\mathbf{U}}{dt} = \mathcal{L}(\mathbf{U}) \tag{37}$$

3.1 Forward Euler Method

The Forward Euler method is a first-order explicit time-integration scheme in which the temporal derivative is approximated using a forward finite difference. The solution at the next time level is expressed as,

$$\boxed{\mathbf{U}_i^{n+1} = \mathbf{U}_i^n + \Delta t\, \mathcal{L}(\mathbf{U}_i^n)} \tag{38}$$

Substituting the finite-volume residual gives,

$$\mathbf{U}_i^{n+1} = \mathbf{U}_i^n - \frac{\Delta t}{\Delta x}\left(\mathbf{F}^n_{i+\frac{1}{2}} - \mathbf{F}^n_{i-\frac{1}{2}}\right) \tag{39}$$

The method requires only the solution at the current time level to advance the solution and is therefore explicit.

3.2 Classical Fourth-Order Runge-Kutta (RK4) Method

The classical fourth-order Runge-Kutta method advances the solution through four intermediate stages to achieve fourth-order accuracy in time. The four stages are calculated as,

$$\begin{aligned}
\mathbf{k}_1 &= \mathcal{L}(\mathbf{U}^n), \\
\mathbf{k}_2 &= \mathcal{L}\left(\mathbf{U}^n + \frac{\Delta t}{2}\mathbf{k}_1\right), \\
\mathbf{k}_3 &= \mathcal{L}\left(\mathbf{U}^n + \frac{\Delta t}{2}\mathbf{k}_2\right), \\
\mathbf{k}_4 &= \mathcal{L}\left(\mathbf{U}^n + \Delta t\,\mathbf{k}_3\right)
\end{aligned} \tag{40}$$

The final solution is obtained using the weighted average of the four stages.

$$\boxed{\mathbf{U}^{n+1} = \mathbf{U}^n + \frac{\Delta t}{6}\left(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4\right)} \tag{41}$$

Unlike Forward Euler, RK4 evaluates the spatial residual at four intermediate solution states during each time step. This provides fourth-order temporal accuracy for sufficiently smooth solutions. Both methods are explicit and are subject to the stability restrictions imposed by the CFL condition.

3.3 CFL-Based Time-Step Calculation

The time-step size is determined dynamically using the Courant-Friedrichs-Lewy (CFL) condition. For the one-dimensional Euler equations, the maximum characteristic wave speed in the computational domain is

$$S_{\max} = \max_i \left(|u_i| + a_i\right) \tag{42}$$

where the local speed of sound is,

$$a_i = \sqrt{\gamma R T_i} \tag{43}$$

The time-step size is thus calculated as,

$$\boxed{\Delta t = \frac{\mathrm{CFL}\,\Delta x}{\max\limits_i(|u_i| + a_i)}} \tag{44}$$

where the CFL number is a prescribed stability parameter and Δx is the uniform spatial grid spacing. The time step is recalculated using the current solution before advancing to the next time level. For all the cases, the domain of $x \in [-1.0,\ 1.0]$ is split into 41 grid cells and $\mathrm{CFL} = 0.2$ is prescribed.

4 Initialisation and Simulation Details

The computational domain is initialised using piecewise-constant primitive variables, with a discontinuity located at $x = 0$. Three initial conditions are implemented to study different flow configurations. The primitive variables are defined as,

$$\mathbf{W} = \begin{Bmatrix} \rho \\ u \\ p \\ T \end{Bmatrix} \tag{45}$$

4.1 Initial Condition 1

A stationary contact discontinuity with different densities and temperatures on either side is considered.

$$\mathbf{W}(x,0) = \begin{cases} \begin{Bmatrix} 1.0 \\ 0 \\ p_L \\ 500 \end{Bmatrix}, & x \leq 0, \\ \begin{Bmatrix} 2.5 \\ 0 \\ p_R \\ 200 \end{Bmatrix}, & x > 0, \end{cases} \tag{46}$$

where the pressures are calculated using the ideal gas equation of state.

$$p = \rho R T \tag{47}$$

4.2 Initial Condition 2

A Mach 3 normal shock configuration is considered. The upstream state is specified using,

$$p_L = 100, \qquad T_L = 401.8, \qquad M_L = 3 \tag{48}$$

The upstream density and velocity are calculated as,

$$\rho_L = \frac{p_L}{R T_L}, \qquad u_L = M_L \sqrt{\gamma R T_L} \tag{49}$$

The downstream state is obtained using the normal shock relations.

$$\frac{p_R}{p_L} = 1 + \frac{2\gamma}{\gamma+1}(M_L^2 - 1), \tag{50}$$

$$\frac{\rho_R}{\rho_L} = \frac{(\gamma+1)M_L^2}{(\gamma-1)M_L^2 + 2} \tag{51}$$

The downstream temperature and velocity are subsequently calculated as,

$$T_R = T_L \frac{p_R/p_L}{\rho_R/\rho_L}, \qquad u_R = \frac{u_L}{\rho_R/\rho_L} \tag{52}$$

4.3 Initial Condition 3

Another stationary discontinuity is considered, with the density and temperature distributions reversed relative to initial condition 1.

$$\mathbf{W}(x,0) = \begin{cases} \begin{Bmatrix} 2.5 \\ 0 \\ p_L \\ 500 \end{Bmatrix}, & x \leq 0, \\ \begin{Bmatrix} 1.0 \\ 0 \\ p_R \\ 250 \end{Bmatrix} & x > 0. \end{cases} \tag{53}$$

The corresponding pressures are again obtained from the ideal gas equation of state.

4.4 Boundary Conditions

Transmissive (zero-gradient) boundary conditions are imposed at both ends of the computational domain using ghost cells. The conservative variables in the ghost cells are assigned the values of their adjacent interior cells.

$$\mathbf{U}_0 = \mathbf{U}_1, \qquad \mathbf{U}_{N+1} = \mathbf{U}_N \tag{54}$$

This allows the flow variables to be extrapolated beyond the computational boundaries without imposing additional gradients.

4.5 Numerical Implementation

The governing equations are discretised using the finite-volume method. The numerical flux at each cell interface is evaluated using one of the five implemented flux schemes.

1. Steger-Warming FVS.
2. Van Leer FVS (local sound speed).
3. Van Leer FVS (common sound speed).
4. AUSM.
5. Roe FDS.

The resulting semi-discrete equations are advanced in time using either Forward Euler or classical RK4 integration. The time-step size is dynamically adjusted according to the CFL condition.

The conservative variables are subsequently converted back into primitive variables to obtain density, velocity, pressure, and temperature distributions.

5 Exact Riemann Solution

The exact solution of the one-dimensional Euler equations is obtained using the analytical solution of the Riemann problem. The initial condition consists of two uniform states separated by a discontinuity at $x = x_0$. The solution evolves into a combination of shock waves, rarefaction waves, and a contact discontinuity. The initial left and right states are specified by,

$$\mathbf{W}(x,0) = \begin{cases} \mathbf{W}_L, & x < x_0, \\ \mathbf{W}_R, & x > x_0 \end{cases} \tag{55}$$

where,

$$\mathbf{W}_L = \begin{Bmatrix} \rho_L \\ u_L \\ p_L \end{Bmatrix}, \qquad \mathbf{W}_R = \begin{Bmatrix} \rho_R \\ u_R \\ p_R \end{Bmatrix} \tag{56}$$

The exact solution is self-similar and depends on the similarity variable,

$$\xi = \frac{x - x_0}{t} \tag{57}$$

Consequently, the solution at any time $t > 0$ can be expressed as a function of ξ rather than independently depending on x and t.

5.1 Pressure Function

The exact Riemann solution is constructed by first determining the pressure and velocity in the star region, denoted by p^* and u^*, respectively. The speed of sound in the initial left and right states is given by,

$$a_L = \sqrt{\frac{\gamma p_L}{\rho_L}}, \qquad a_R = \sqrt{\frac{\gamma p_R}{\rho_R}} \tag{58}$$

A pressure function $\phi_k(p)$ is defined for each initial state $k = L, R$ as,

$$\phi_k(p) = \begin{cases} (p - p_k)\sqrt{\dfrac{2}{(\gamma+1)\rho_k}\dfrac{1}{p + \dfrac{\gamma-1}{\gamma+1}p_k}}, & p > p_k, \\ \dfrac{2a_k}{\gamma-1}\left[\left(\dfrac{p}{p_k}\right)^{\frac{\gamma-1}{2\gamma}} - 1\right], & p \le p_k \end{cases} \tag{59}$$

The first expression corresponds to a shock wave, while the second corresponds to a rarefaction wave. The pressure in the star region is obtained by solving the nonlinear equation

$$\boxed{\phi_L(p^*) + \phi_R(p^*) + u_R - u_L = 0} \tag{60}$$

This nonlinear equation is solved numerically using the bisection method. The initial pressure interval is chosen as,

$$p_{\text{lo}} = 0, \qquad p_{\text{hi}} = \max(p_L, p_R) \tag{61}$$

The upper bound is repeatedly doubled until the pressure equation changes sign over the interval. The bisection procedure is then performed until the prescribed convergence tolerance is reached or the maximum number of iterations is exhausted.

5.2 Star Region Properties

Once the star-region pressure p^* is determined, the corresponding velocity is calculated as

$$\boxed{u^* = \frac{1}{2}\left[u_L + u_R + \phi_R(p^*) - \phi_L(p^*)\right]} \tag{62}$$

The density in the left and right star regions depends on whether the corresponding wave is a shock or a rarefaction. For a shock wave, where $p^* > p_k$, the star-region density is obtained from the Rankine-Hugoniot relations.

$$\boxed{\rho_k^* = \rho_k \frac{\dfrac{p^*}{p_k} + \dfrac{\gamma-1}{\gamma+1}}{\dfrac{\gamma-1}{\gamma+1}\dfrac{p^*}{p_k} + 1}} \tag{63}$$

For a rarefaction wave, where $p^* \le p_k$, the isentropic relation is used,

$$\boxed{\rho_k^* = \rho_k \left(\frac{p^*}{p_k}\right)^{1/\gamma}} \tag{64}$$

The corresponding speed of sound in the star region for a rarefaction wave is,

$$a_k^* = a_k \left(\frac{p^*}{p_k}\right)^{\frac{\gamma-1}{2\gamma}} \tag{65}$$

The pressure and velocity are uniform across the contact discontinuity, whereas the density may be different on either side.

$$p_L^* = p_R^* = p^*, \qquad u_L^* = u_R^* = u^* \tag{66}$$

5.3 Wave Speeds

The wave structure on each side of the contact discontinuity is determined by comparing the star-region pressure with the corresponding initial pressure.

Left Shock Wave

When $p^* > p_L$, the left wave is a shock wave. Its propagation speed is given by,

$$\boxed{S_L = u_L - a_L\sqrt{\frac{\gamma+1}{2\gamma}\frac{p^*}{p_L} + \frac{\gamma-1}{2\gamma}}} \tag{67}$$

Left Rarefaction Wave

When $p^* \leq p_L$, the left wave is a rarefaction fan. The head and tail velocities of the fan are,

$$S_{HL} = u_L - a_L, \qquad S_{TL} = u^* - a_L^* \tag{68}$$

Right Shock Wave

When $p^* > p_R$, the right wave is a shock wave. Its propagation speed is,

$$\boxed{S_R = u_R + a_R\sqrt{\frac{\gamma+1}{2\gamma}\frac{p^*}{p_R} + \frac{\gamma-1}{2\gamma}}} \tag{69}$$

Right Rarefaction Wave

When $p^* \leq p_R$, the right wave is a rarefaction fan. The head and tail velocities are,

$$S_{HR} = u_R + a_R, \qquad S_{TR} = u^* + a_R^* \tag{70}$$

5.4 Solution Inside the Rarefaction Fans

Within a rarefaction fan, the flow variables vary continuously and are obtained from the self-similar solution of the Euler equations. For the left rarefaction fan, the velocity and local speed of sound are given by,

$$u(\xi) = \frac{2}{\gamma+1}\left[a_L + \frac{\gamma-1}{2}u_L + \xi\right] \tag{71}$$

$$a(\xi) = \frac{2}{\gamma+1}\left[a_L + \frac{\gamma-1}{2}(u_L - \xi)\right] \tag{72}$$

The density and pressure are subsequently obtained using the isentropic relations.

$$\rho(\xi) = \rho_L \left(\frac{a(\xi)}{a_L}\right)^{\frac{2}{\gamma-1}} \tag{73}$$

$$p(\xi) = p_L \left(\frac{a(\xi)}{a_L}\right)^{\frac{2\gamma}{\gamma-1}} \tag{74}$$

Similarly, for the right rarefaction fan, the velocity and local speed of sound are,

$$u(\xi) = \frac{2}{\gamma+1}\left[-a_R + \frac{\gamma-1}{2}u_R + \xi\right] \tag{75}$$

$$a(\xi) = \frac{2}{\gamma+1}\left[a_R - \frac{\gamma-1}{2}(u_R - \xi)\right] \tag{76}$$

The corresponding density and pressure are,

$$\rho(\xi) = \rho_R \left(\frac{a(\xi)}{a_R}\right)^{\frac{2}{\gamma-1}} \tag{77}$$

$$p(\xi) = p_R \left(\frac{a(\xi)}{a_R}\right)^{\frac{2\gamma}{\gamma-1}} \tag{78}$$

5.5 Evaluation of the Exact Solution

After calculating the star-region properties and wave speeds, the solution at each spatial location is determined using the similarity variable $\xi = (x - x_0)/t$. On the left side of the contact discontinuity, the solution is classified as follows. For a left shock wave,

$$\mathbf{W}(\xi) = \begin{cases} \mathbf{W}_L, & \xi \le S_L, \\ \mathbf{W}_L^*, & S_L < \xi \le u^*. \end{cases} \tag{79}$$

For a left rarefaction wave,

$$\mathbf{W}(\xi) = \begin{cases} \mathbf{W}_L, & \xi \le S_{HL}, \\ \mathbf{W}_{\text{fan},L}(\xi), & S_{HL} < \xi < S_{TL}, \\ \mathbf{W}_L^*, & S_{TL} \le \xi \le u^*. \end{cases} \tag{80}$$

Similarly, on the right side of the contact discontinuity, for a right shock wave,

$$\mathbf{W}(\xi) = \begin{cases} \mathbf{W}_R^*, & u^* < \xi < S_R, \\ \mathbf{W}_R, & \xi \ge S_R. \end{cases} \tag{81}$$

For a right rarefaction wave,

$$\mathbf{W}(\xi) = \begin{cases} \mathbf{W}_R^*, & u^* < \xi \leq S_{TR}, \\ \mathbf{W}_{\text{fan},R}(\xi), & S_{TR} < \xi < S_{HR}, \\ \mathbf{W}_R, & \xi \geq S_{HR}. \end{cases} \tag{82}$$

Here, $\mathbf{W}_{\text{fan},L}$ and $\mathbf{W}_{\text{fan},R}$ denote the continuously varying solutions inside the left and right rarefaction fans, respectively. The temperature is recovered from the ideal gas equation of state.

$$T(\xi) = \frac{p(\xi)}{\rho(\xi)R}, \tag{83}$$

where R is the specific gas constant. At the initial time, $t = 0$, the solution is assigned directly from the initial left and right states.

5.6 Implementation

The exact Riemann solver is implemented as a separate Fortran module. The star-region pressure is calculated using a bisection procedure with a maximum of 100 iterations and a convergence tolerance of 10^{-12}.

For each spatial location, the similarity variable is evaluated and used to identify the corresponding wave region. The density, velocity, and pressure are then calculated using the appropriate constant-state or rarefaction-fan expressions. Temperature is subsequently recovered using the ideal gas equation of state.

The resulting exact solution provides a reference for validating the numerical solutions obtained using the different numerical flux and time-integration schemes.

6 Results and Discussion

The numerical solutions obtained using different numerical flux schemes and time integration methods are compared with the corresponding exact Riemann solutions. The comparison is performed for three initial conditions, considering five numerical flux schemes and two time integration methods.

The results presented in this section correspond to the final simulation time, $t = 0.0020$. Each figure shows the spatial distributions of density (ρ), velocity (u), pressure (p), and temperature (T). The numerical solutions are compared against the exact solutions, which serve as the reference for assessing the accuracy of the numerical methods.

6.1 Initial Condition 1: Stationary Contact Discontinuity

The first initial condition represents a stationary contact discontinuity. The initial states have equal pressure and velocity, but different densities and temperatures. Consequently, the discontinuity should

remain stationary in the absence of numerical errors.

6.1.1 Steger-Warming Flux Scheme

6.1.1.1 Forward Euler Time Integration

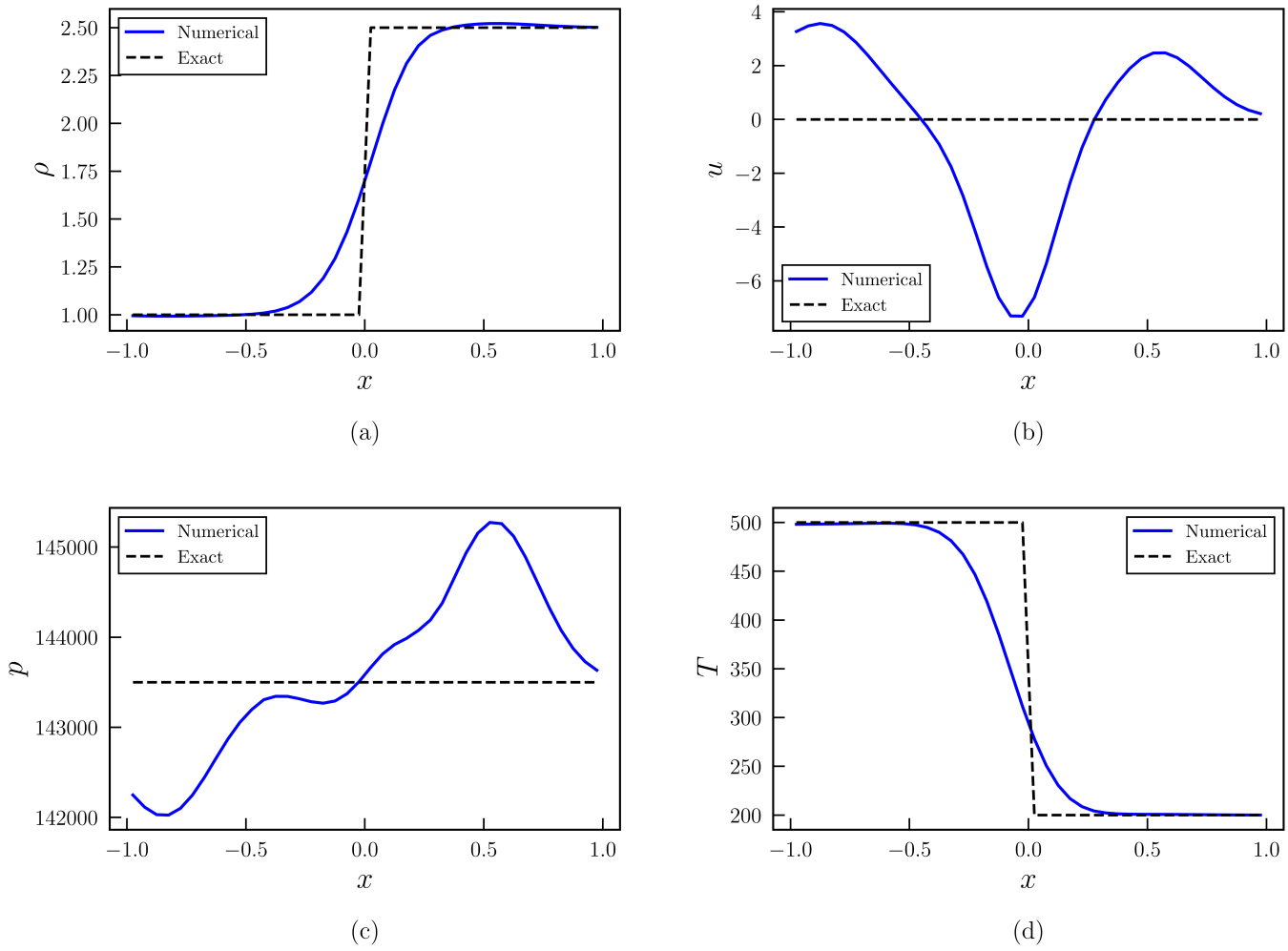


Figure 1: Comparison of numerical and exact solutions for Initial Condition 1 using the Steger-Warming flux scheme with Forward Euler time integration at $t = 0.0020$.

6.1.1.2 Runge-Kutta 4 Time Integration

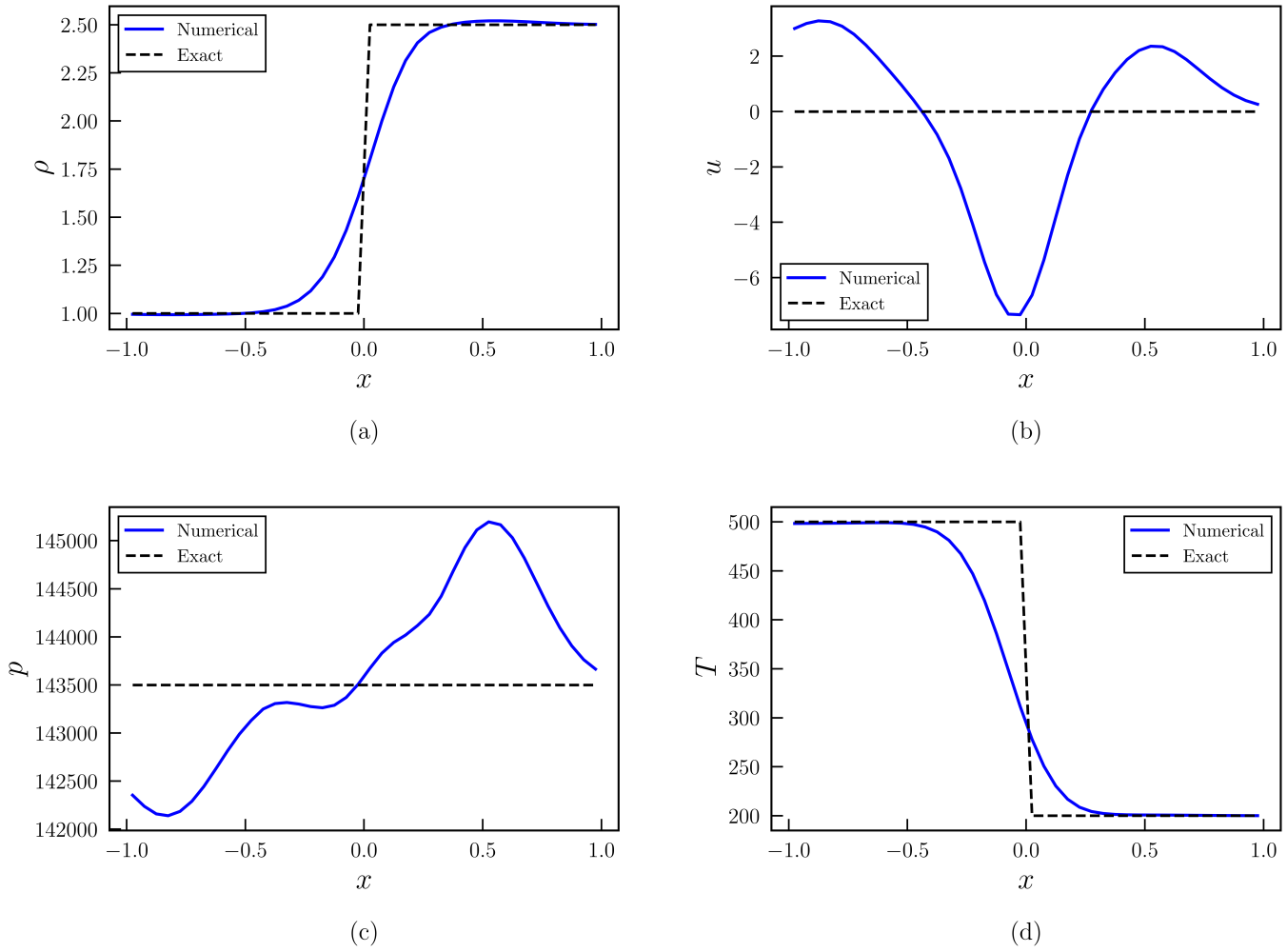


Figure 2: Comparison of numerical and exact solutions for Initial Condition 1 using the Steger-Warming flux scheme with classical fourth-order Runge-Kutta (RK4) time integration at $t = 0.0020$.

Both time integration schemes produce similar results. The stationary contact discontinuity is smeared in density and temperature, and the pressure and velocites are not preserved.

6.1.2 Van Leer (Local) Flux Scheme

6.1.2.1 Forward Euler Time Integration

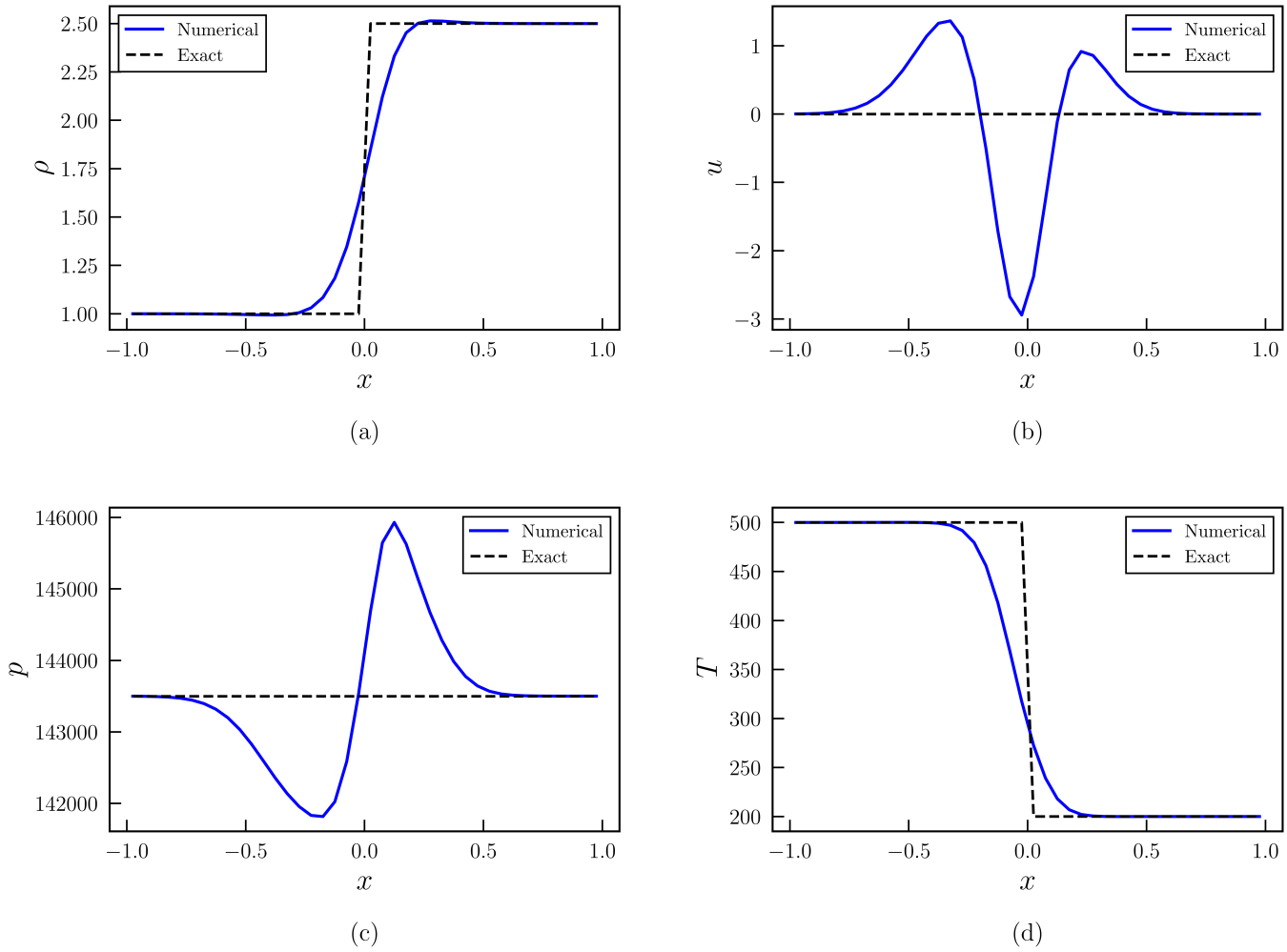


Figure 3: Comparison of numerical and exact solutions for Initial Condition 1 using the Van Leer (Local) flux scheme with Forward Euler time integration at $t = 0.0020$.

6.1.2.2 Runge-Kutta 4 Time Integration

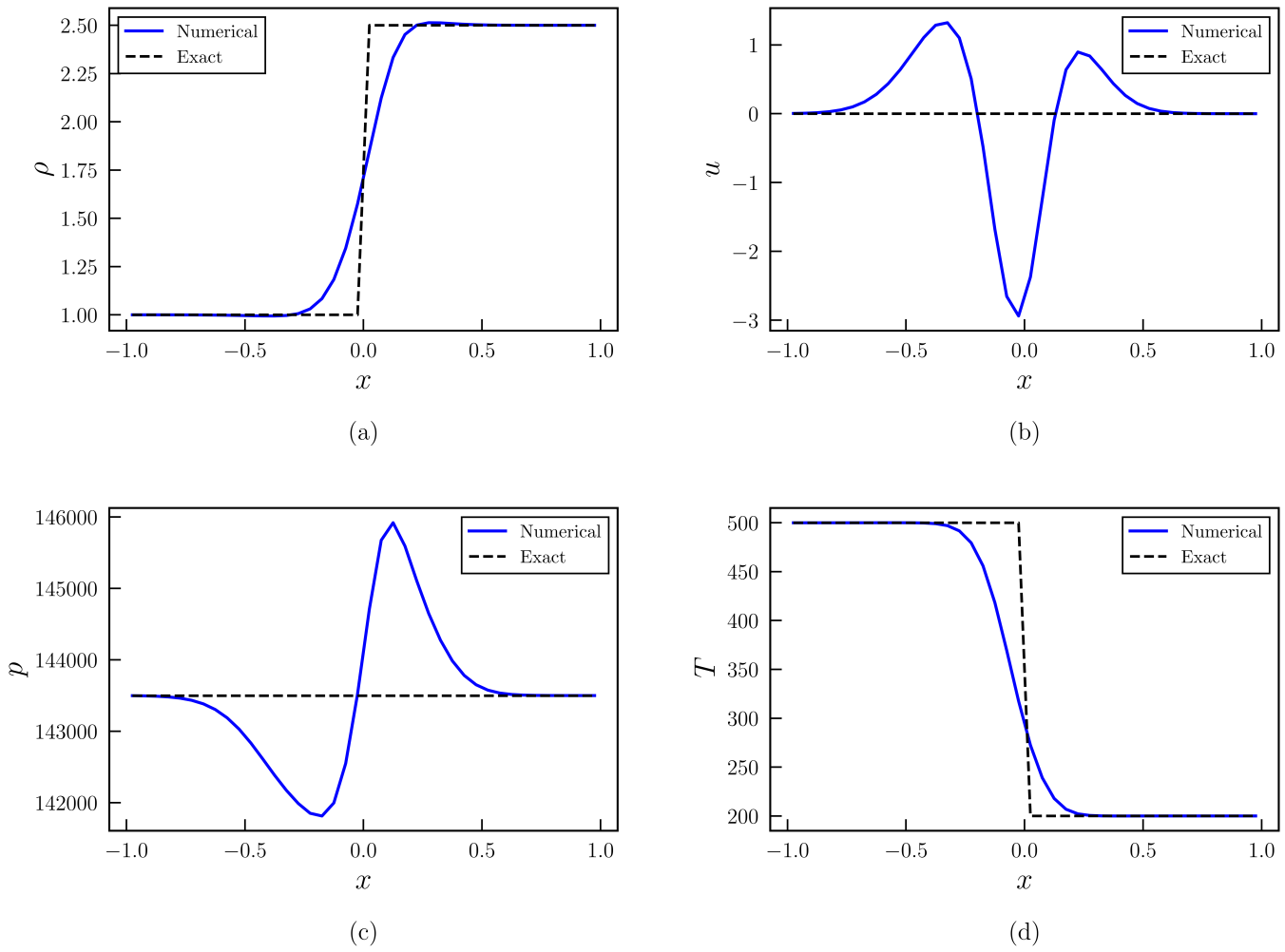


Figure 4: Comparison of numerical and exact solutions for Initial Condition 1 using the Van Leer (Local) flux scheme with RK4 time integration at $t = 0.0020$.

A similar smearing of the discontinous fields is observed but considerably less smearing is observed compared to the Steger-Warming scheme. Both timestepping methods produce similar results.

6.1.3 Van Leer (Common) Flux Scheme

6.1.3.1 Forward Euler Time Integration

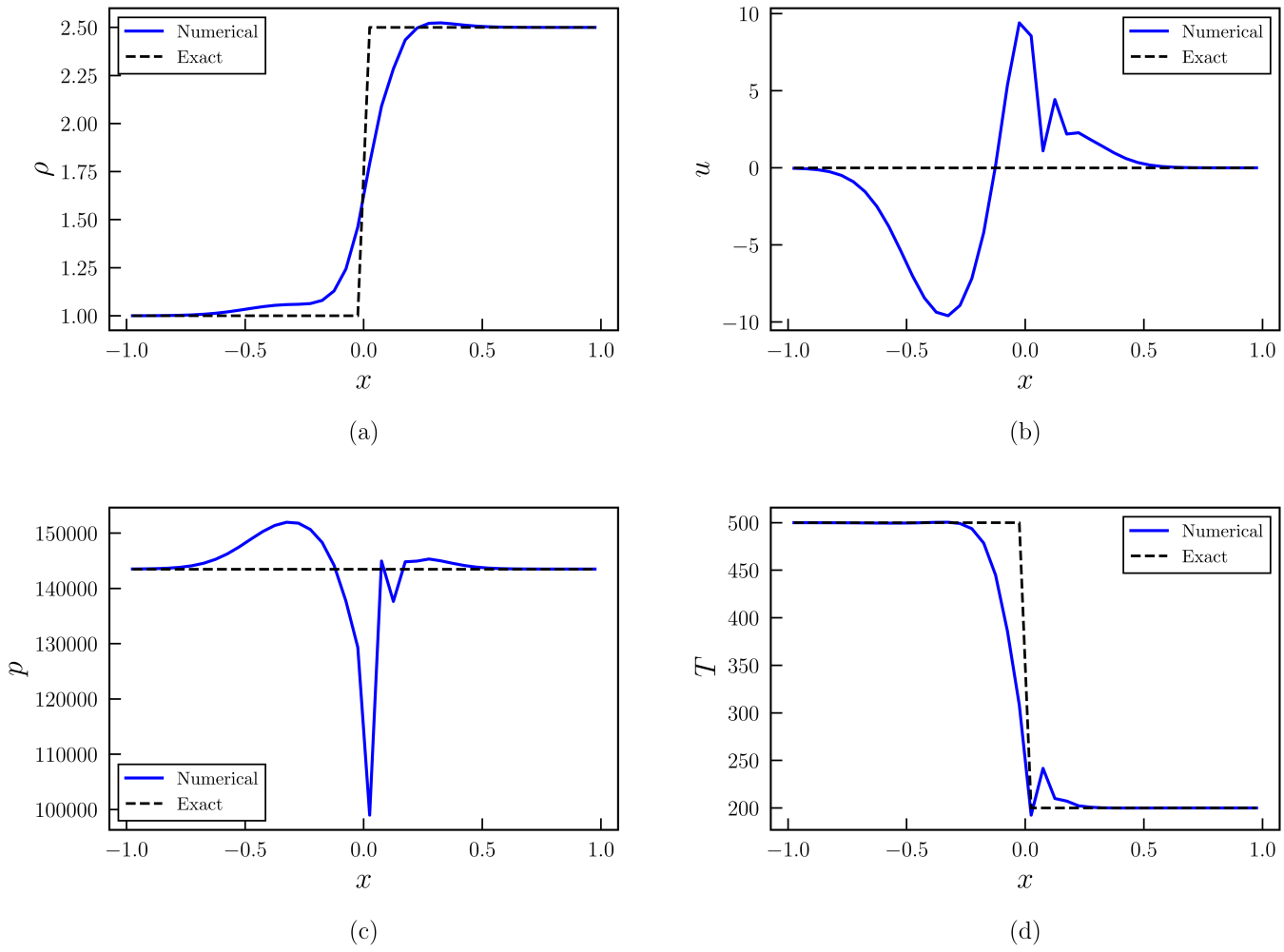


Figure 5: Comparison of numerical and exact solutions for Initial Condition 1 using the Van Leer (Common) flux scheme with Forward Euler time integration at $t = 0.0020$.

6.1.3.2 Runge-Kutta 4 Time Integration

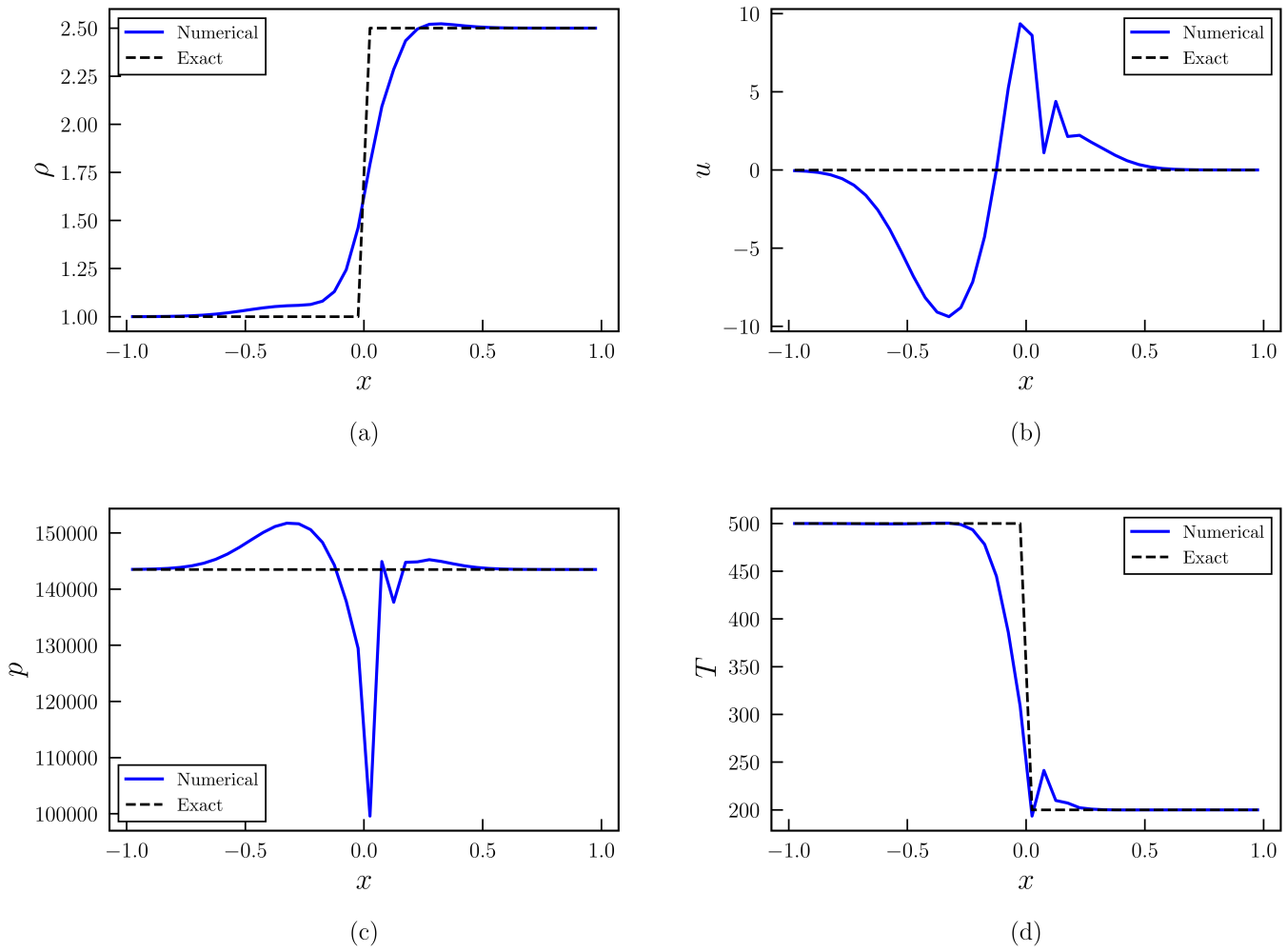


Figure 6: Comparison of numerical and exact solutions for Initial Condition 1 using the Van Leer (Common) flux scheme with RK4 time integration at $t = 0.0020$.

The discontinuity is smeared here as well, but in this case the value is lost much earlier in density and a spike is observed in pressure. There is no difference in terms of the timestepping schemes.

6.1.4 AUSM Flux Scheme

6.1.4.1 Forward Euler Time Integration

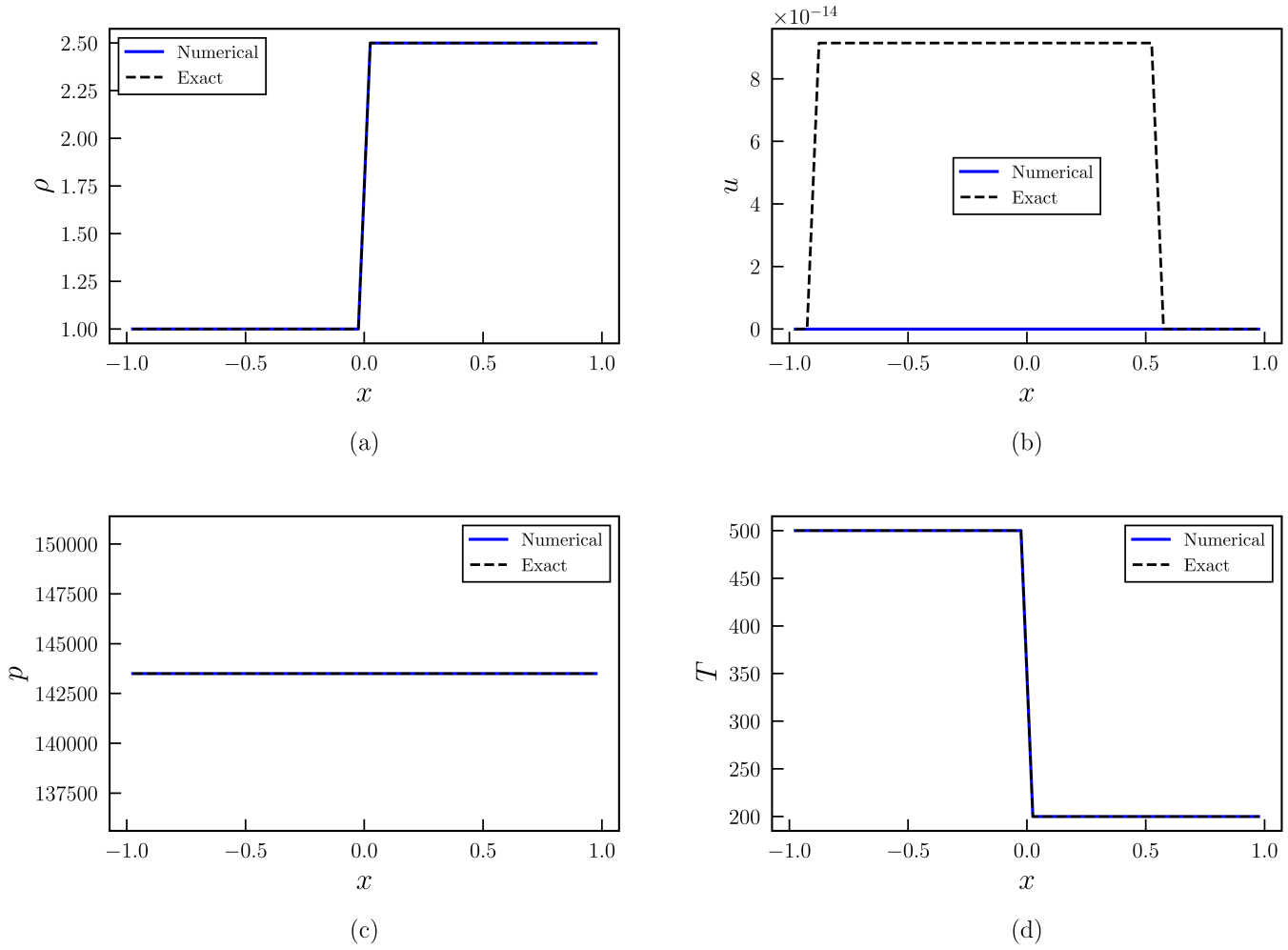


Figure 7: Comparison of numerical and exact solutions for Initial Condition 1 using the AUSM flux scheme with Forward Euler time integration at $t = 0.0020$.

6.1.4.2 Runge-Kutta 4 Time Integration

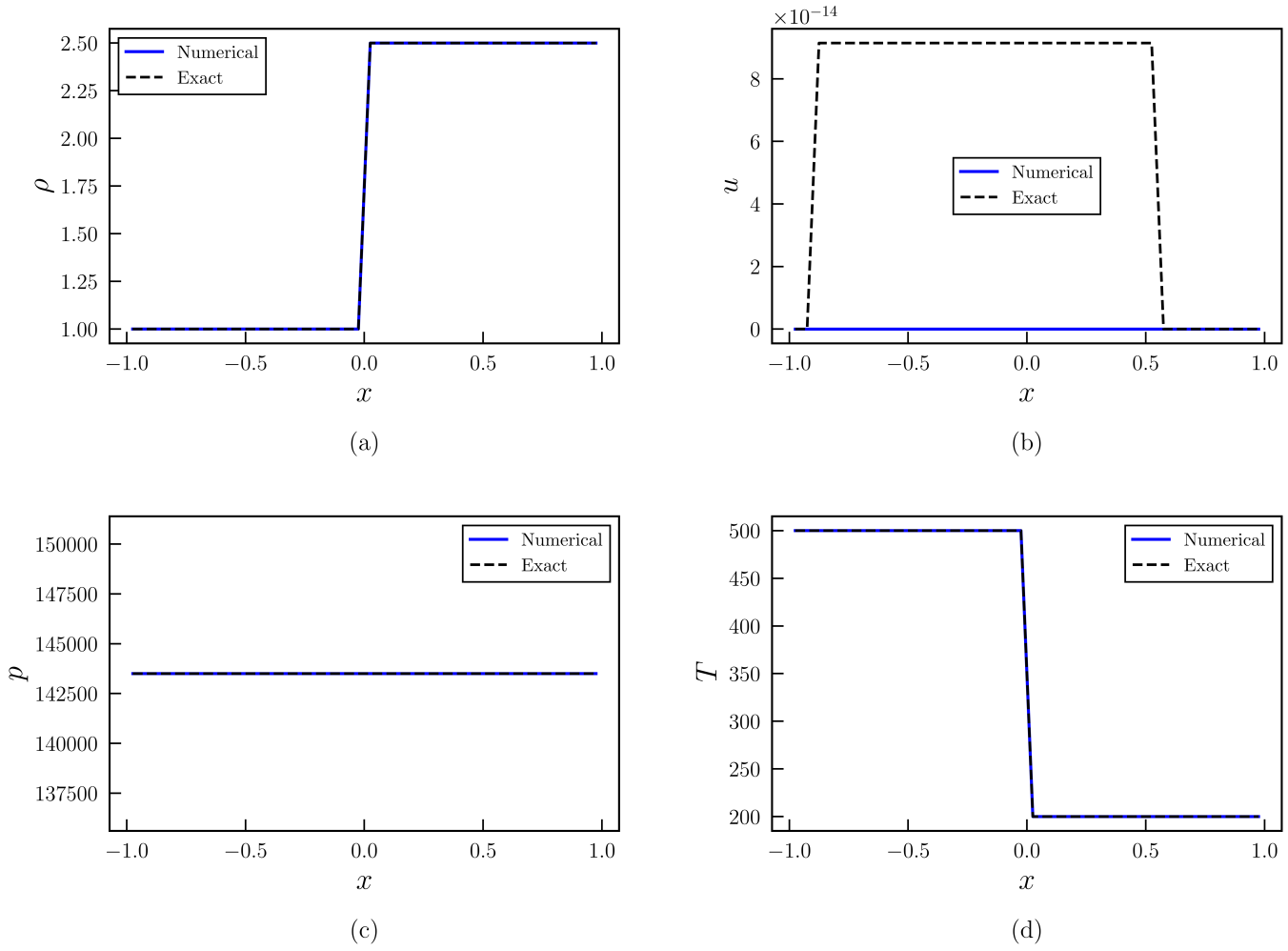


Figure 8: Comparison of numerical and exact solutions for Initial Condition 1 using the AUSM flux scheme with RK4 time integration at $t = 0.0020$.

The AUSM scheme captures the exact solution very well. The difference in the velocity is of the order of machine epcilon. Both time integration schemes produce the same result.

6.1.5 Roe Flux Scheme

6.1.5.1 Forward Euler Time Integration

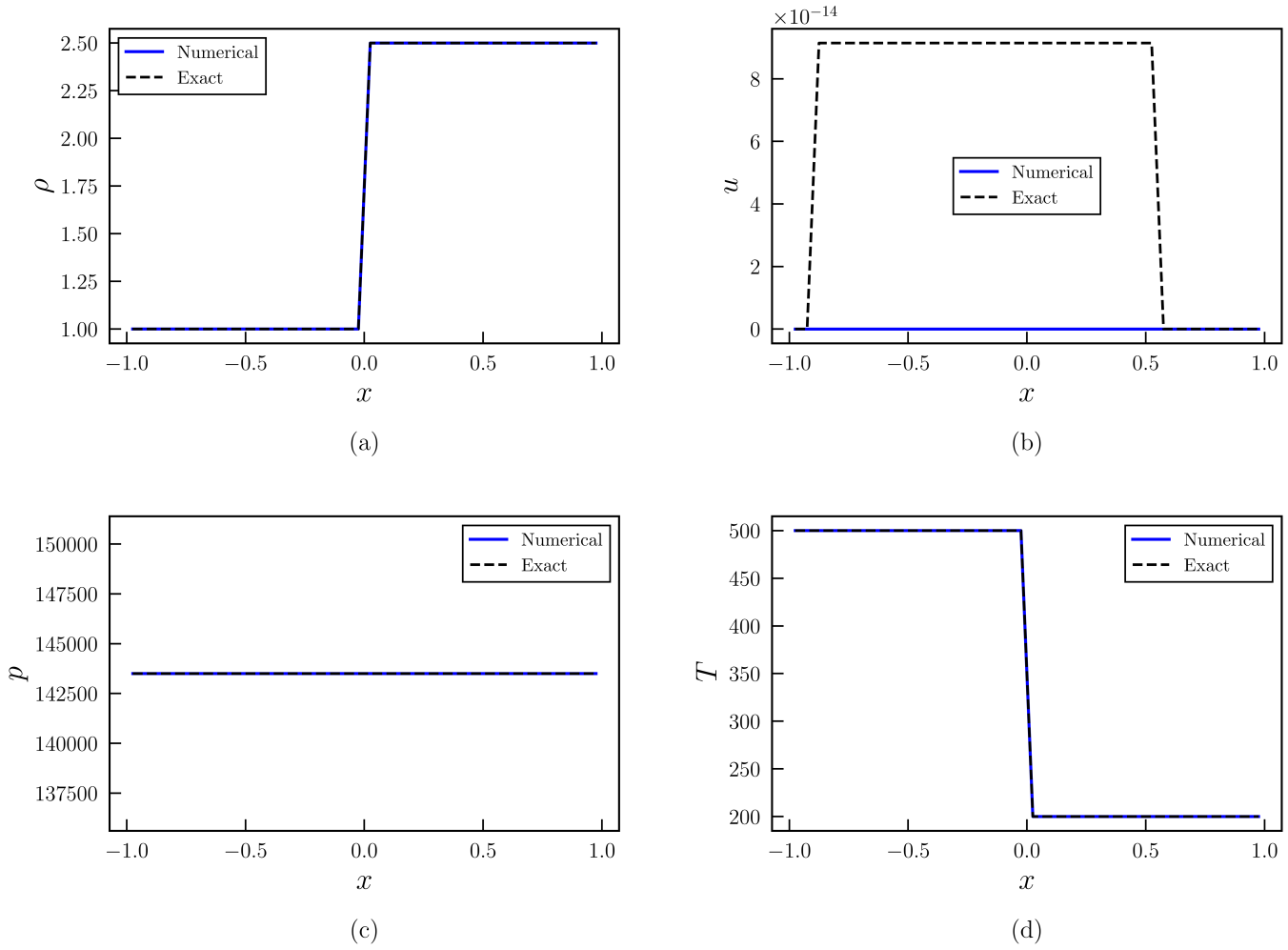


Figure 9: Comparison of numerical and exact solutions for Initial Condition 1 using the Roe flux scheme with Forward Euler time integration at $t = 0.0020$.

6.1.5.2 Runge-Kutta 4 Time Integration

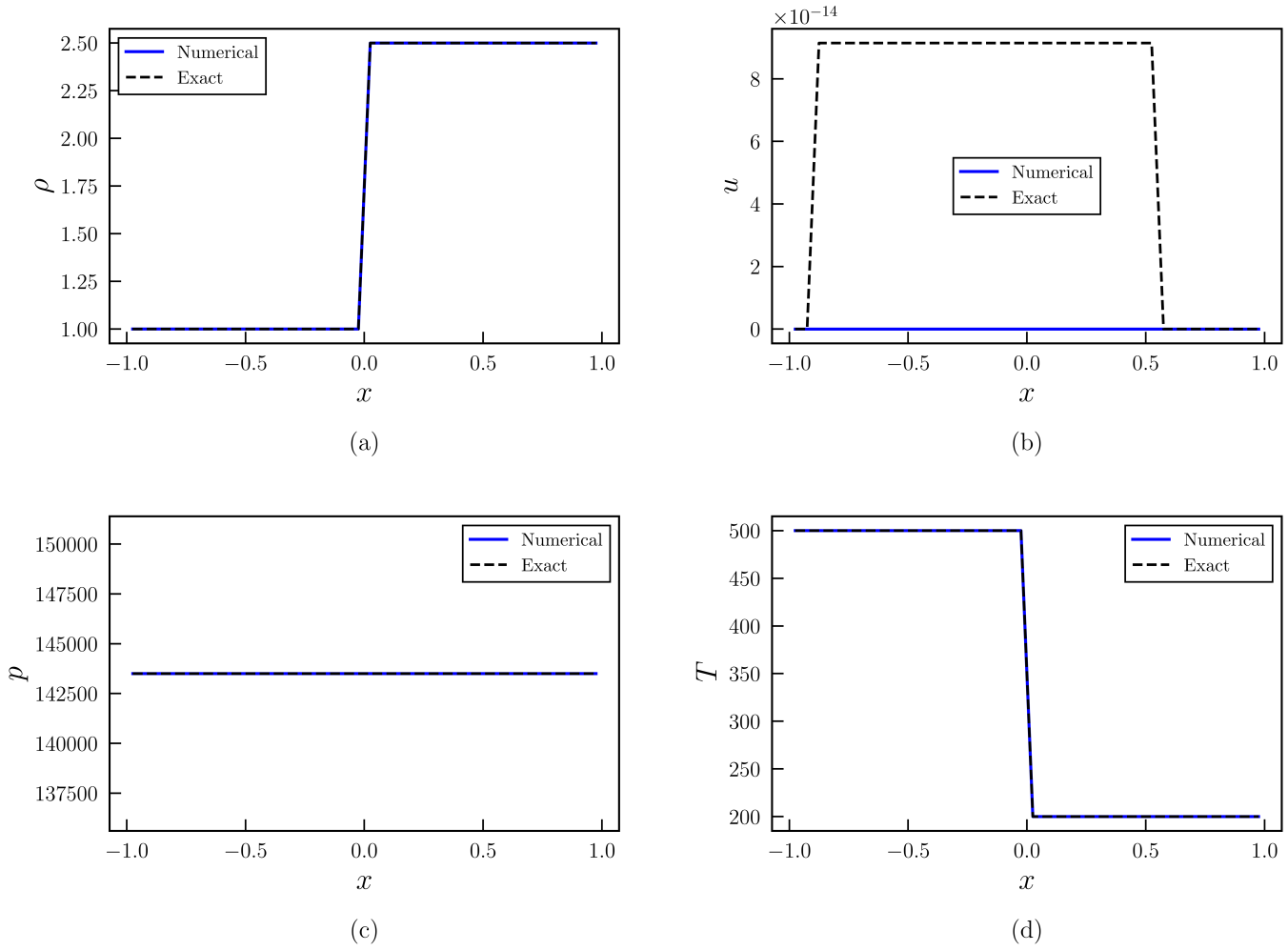


Figure 10: Comparison of numerical and exact solutions for Initial Condition 1 using the Roe flux scheme with RK4 time integration at $t = 0.0020$.

Similar to the AUSM scheme, the Roe scheme is also able to capture the stationary contact discontinuity accurately. No effect of the time integration scheme is observed, both results are identical.

6.2 Initial Condition 2: Stationary Normal Shock

The second initial condition represents a stationary normal shock. The upstream and downstream states are constructed using the Rankine-Hugoniot relations such that the shock remains stationary. The numerical results are therefore expected to preserve the initial discontinuity.

6.2.1 Steger–Warming Flux Scheme

6.2.1.1 Forward Euler Time Integration

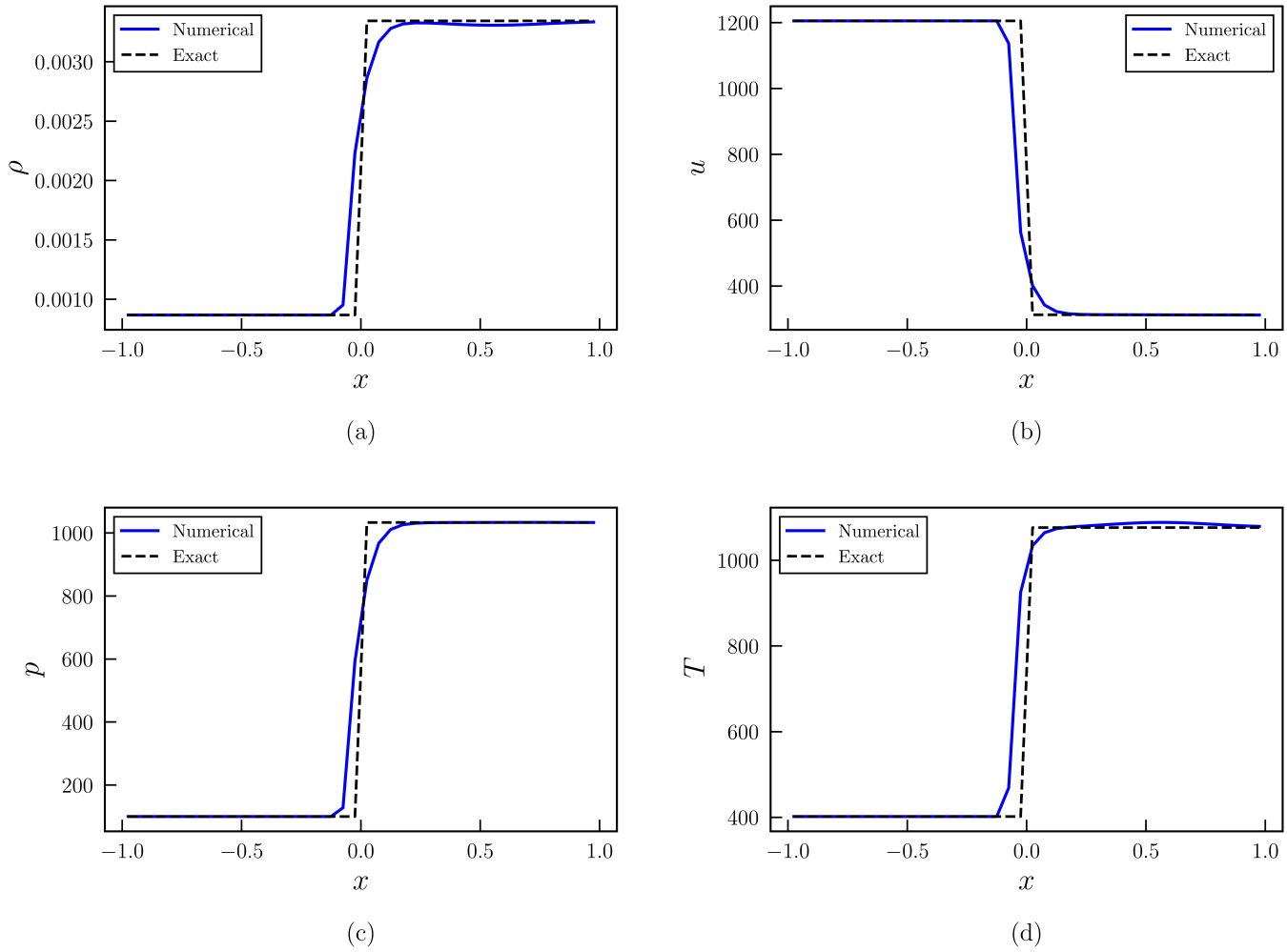


Figure 11: Comparison of numerical and exact solutions for Initial Condition 2 using the Steger–Warming flux scheme with Forward Euler time integration at $t = 0.0020$.

6.2.1.2 Runge-Kutta 4 Time Integration

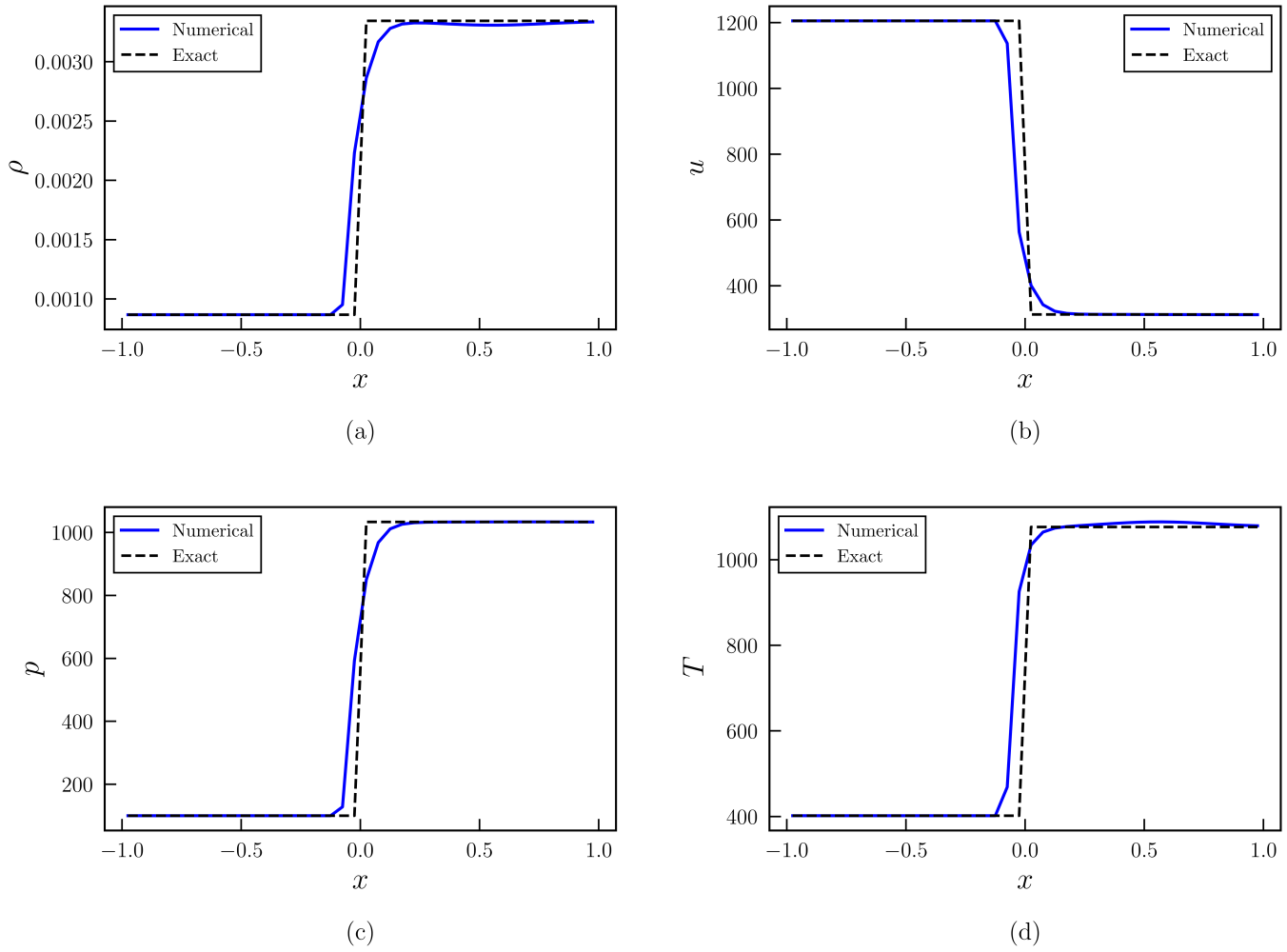


Figure 12: Comparison of numerical and exact solutions for Initial Condition 2 using the Steger–Warming flux scheme with RK4 time integration at $t = 0.0020$.

No effect of the timestepping scheme is observed. Both results exhibit some deviation form the exact solution, this can be attributed to grid resolution. Essentially, the normal shock si slightly smeared.

6.2.2 Van Leer (Local) Flux Scheme

6.2.2.1 Forward Euler Time Integration

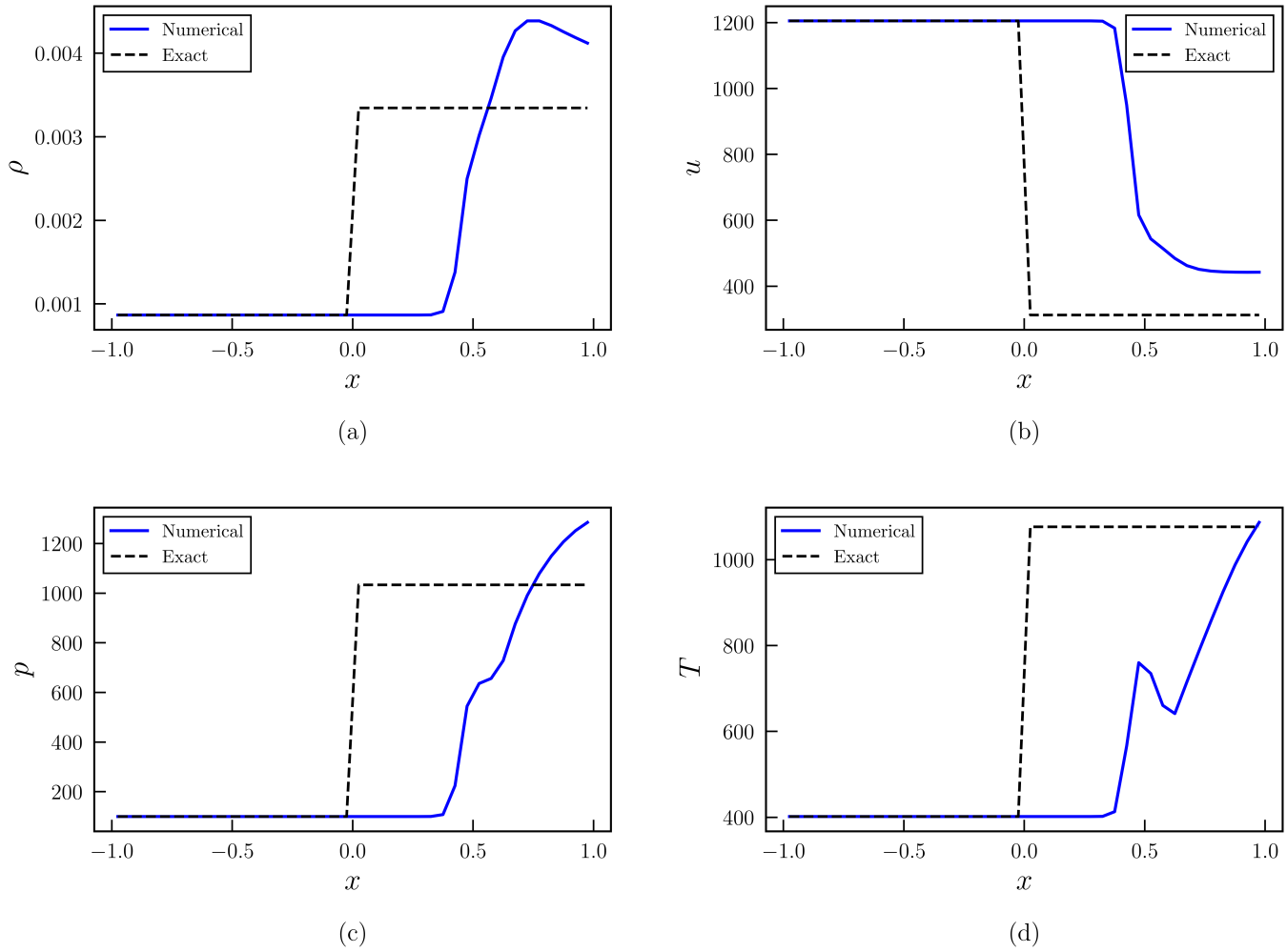


Figure 13: Comparison of numerical and exact solutions for Initial Condition 2 using the Van Leer (Local) flux scheme with Forward Euler time integration at $t = 0.0020$.

6.2.2.2 Runge-Kutta 4 Time Integration

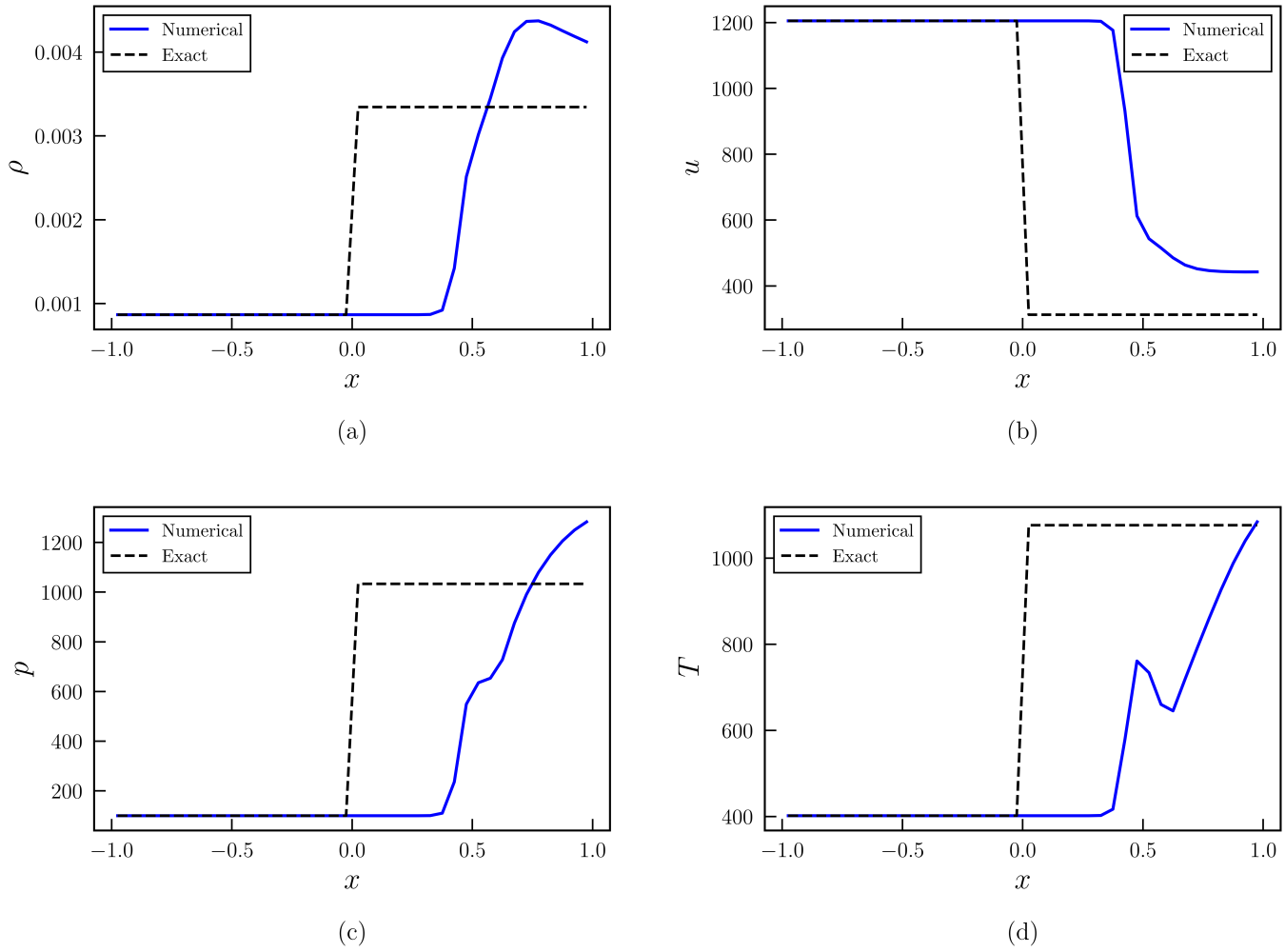


Figure 14: Comparison of numerical and exact solutions for Initial Condition 2 using the Van Leer (Local) flux scheme with RK4 time integration at $t = 0.0020$.

This scheme is unable to capture the normal shock in both timestepping cases. It completely messes up the normal schock.

6.2.3 Van Leer (Common) Flux Scheme

The final-time results for this configuration resulted in negative pressure and did not run. This could be attributed to the grid resolution.

6.2.4 AUSM Flux Scheme

6.2.4.1 Forward Euler Time Integration

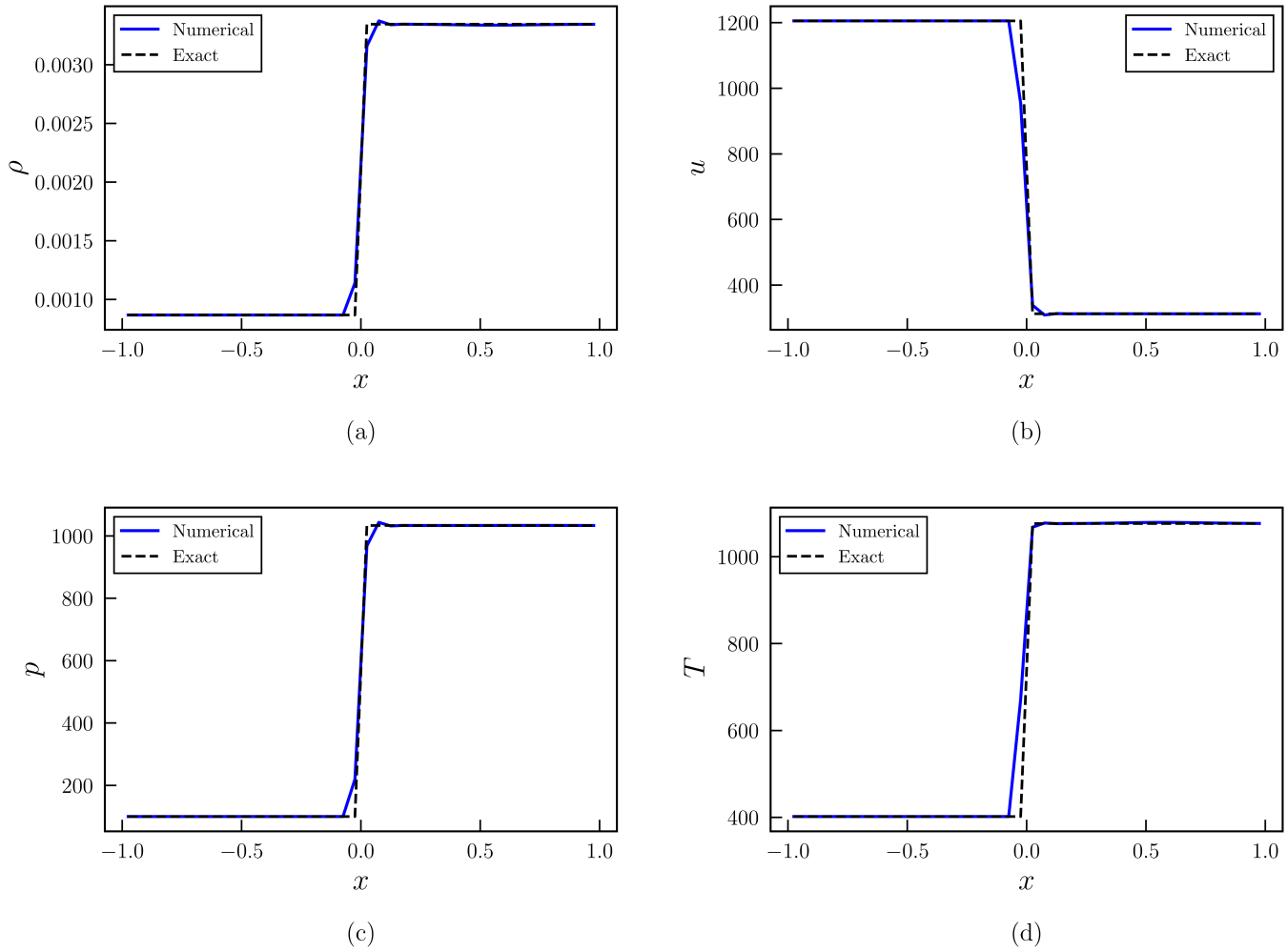


Figure 15: Comparison of numerical and exact solutions for Initial Condition 2 using the AUSM flux scheme with Forward Euler time integration at $t = 0.0020$.

6.2.4.2 Runge-Kutta 4 Time Integration

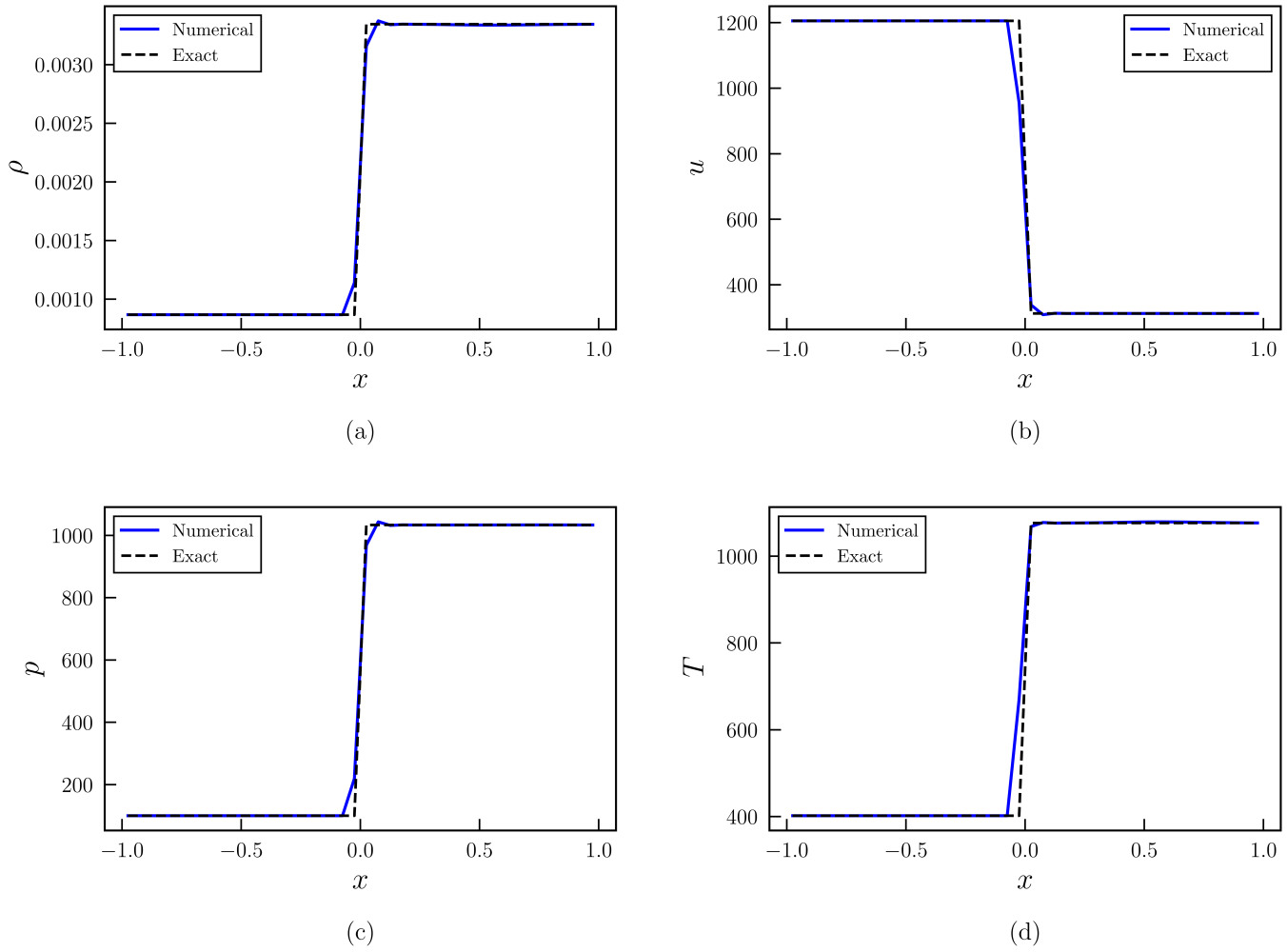


Figure 16: Comparison of numerical and exact solutions for Initial Condition 2 using the AUSM flux scheme with RK4 time integration at $t = 0.0020$.

The AUSM scheme does a much better job at capturing the normal shock than the Steger-Warming scheme. No change in the solution is observed upon changing the timestepping scheme.

6.2.5 Roe Flux Scheme

6.2.5.1 Forward Euler Time Integration

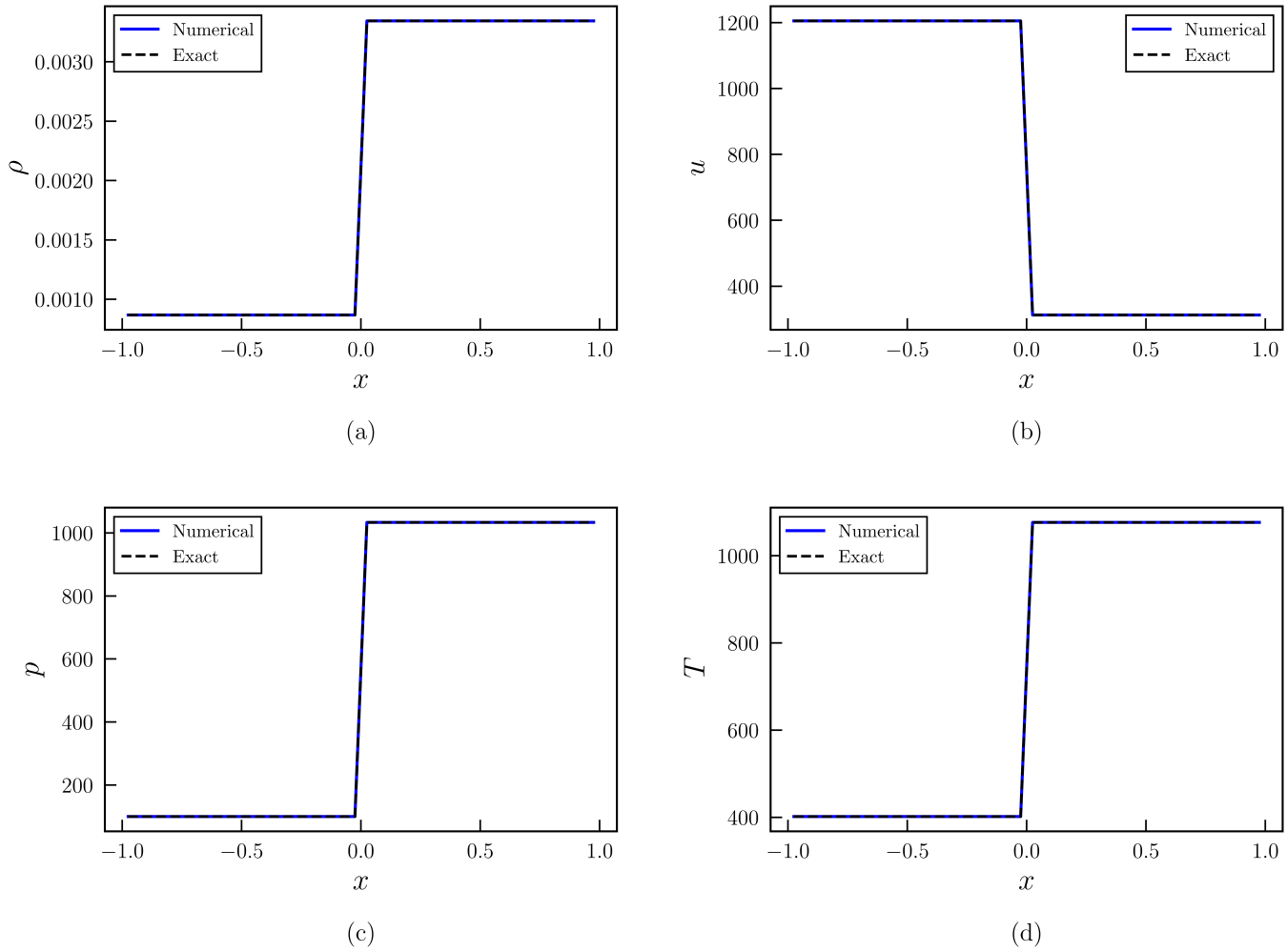


Figure 17: Comparison of numerical and exact solutions for Initial Condition 2 using the Roe flux scheme with Forward Euler time integration at $t = 0.0020$.

6.2.5.2 Runge-Kutta 4 Time Integration

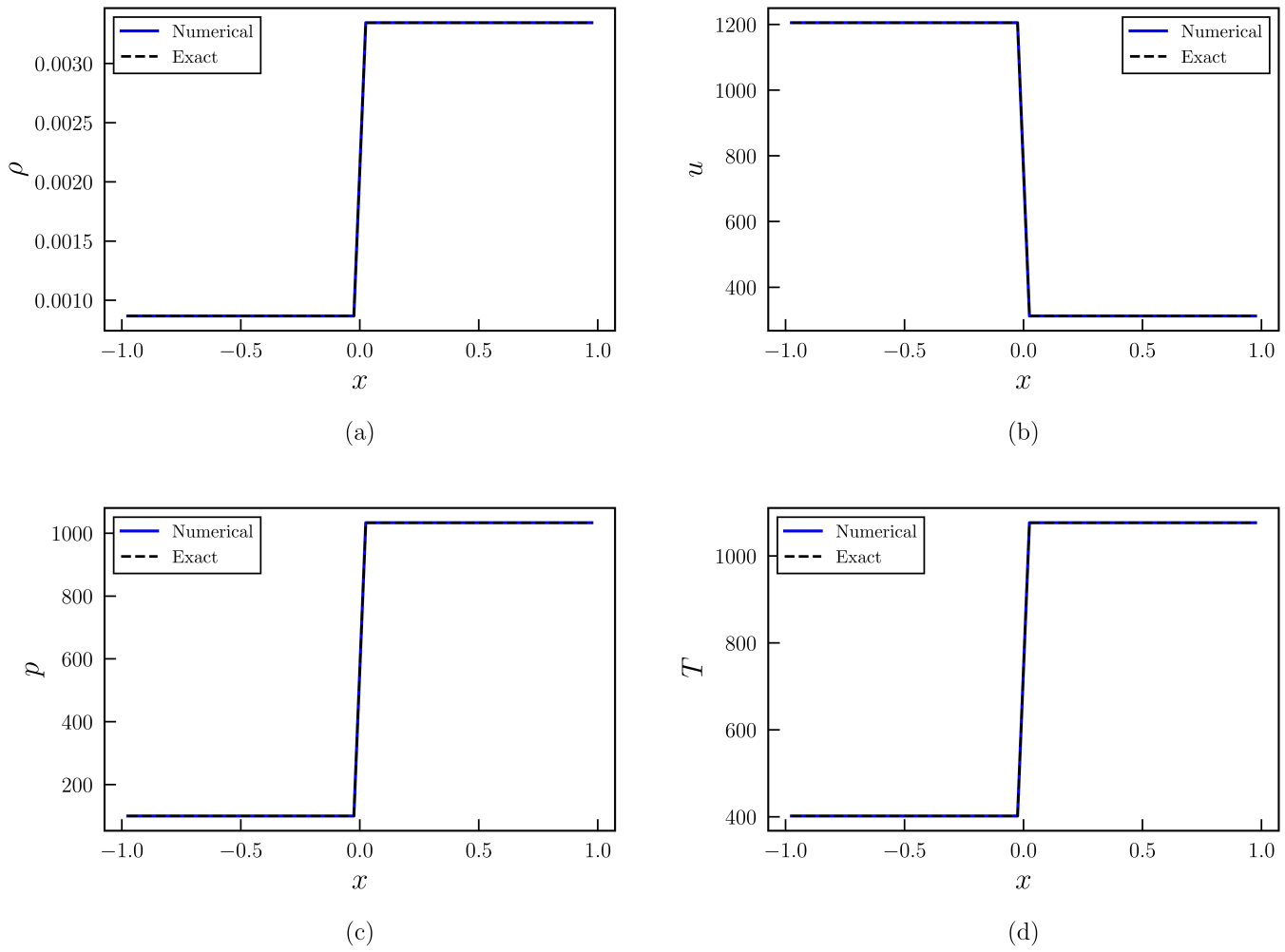


Figure 18: Comparison of numerical and exact solutions for Initial Condition 2 using the Roe flux scheme with RK4 time integration at $t = 0.0020$.

The Roe flux scheme is able to capture the stationary normal shock exactly. No effect of the timestepper is observed.

6.3 Initial Condition 3: Rarefaction-Contact-Shock Interaction

The third initial condition produces a complex wave interaction involving a left-propagating rarefaction wave, a contact discontinuity, and a right-propagating shock wave. Unlike the first two cases, this initial condition generates a time-dependent solution and provides a useful test of the ability of the numerical schemes to capture different wave structures.

6.3.1 Steger-Warming Flux Scheme

6.3.1.1 Forward Euler Time Integration

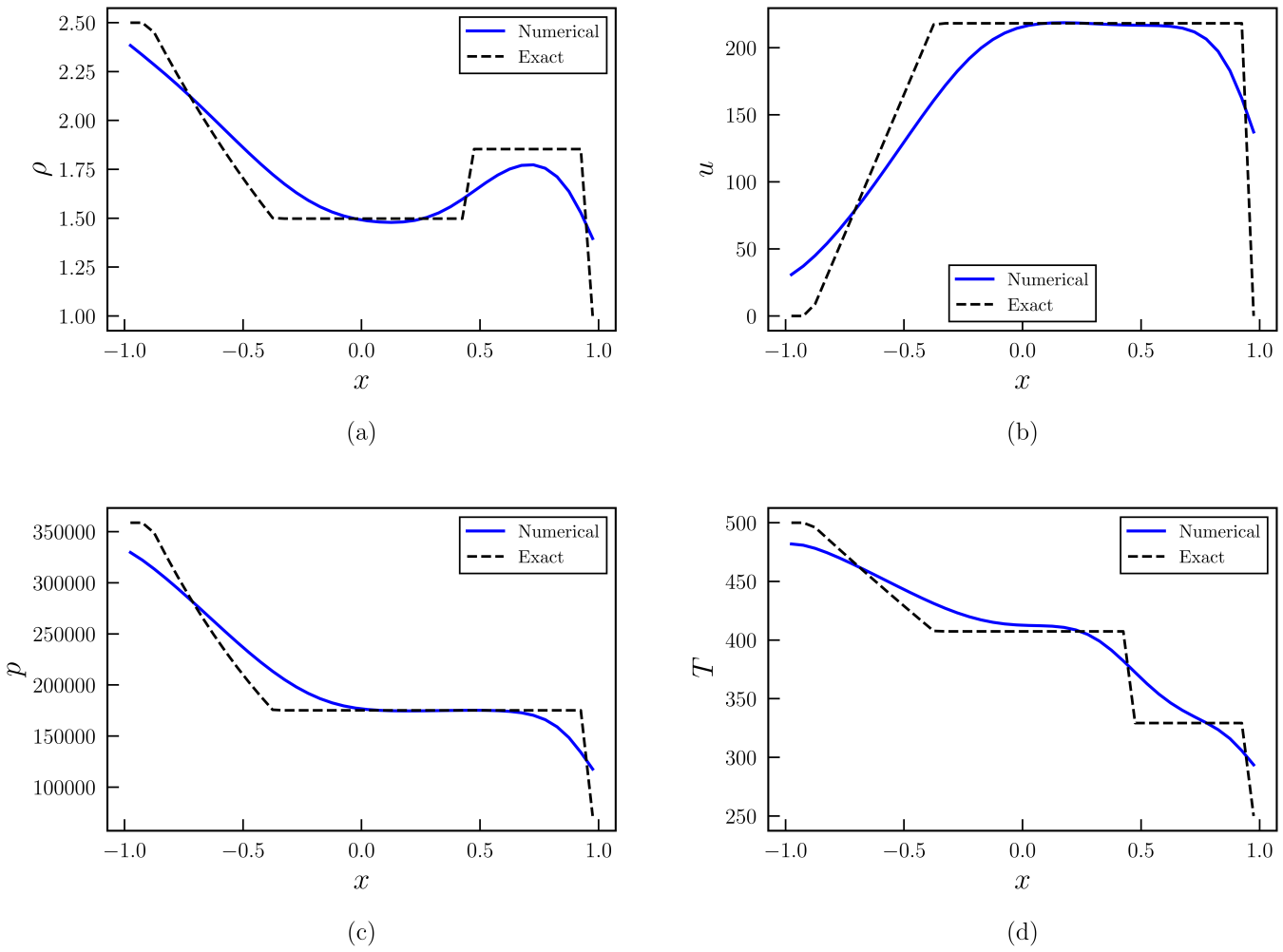


Figure 19: Comparison of numerical and exact solutions for Initial Condition 3 using the Steger–Warming flux scheme with Forward Euler time integration at $t = 0.0020$.

6.3.1.2 Runge-Kutta 4 Time Integration

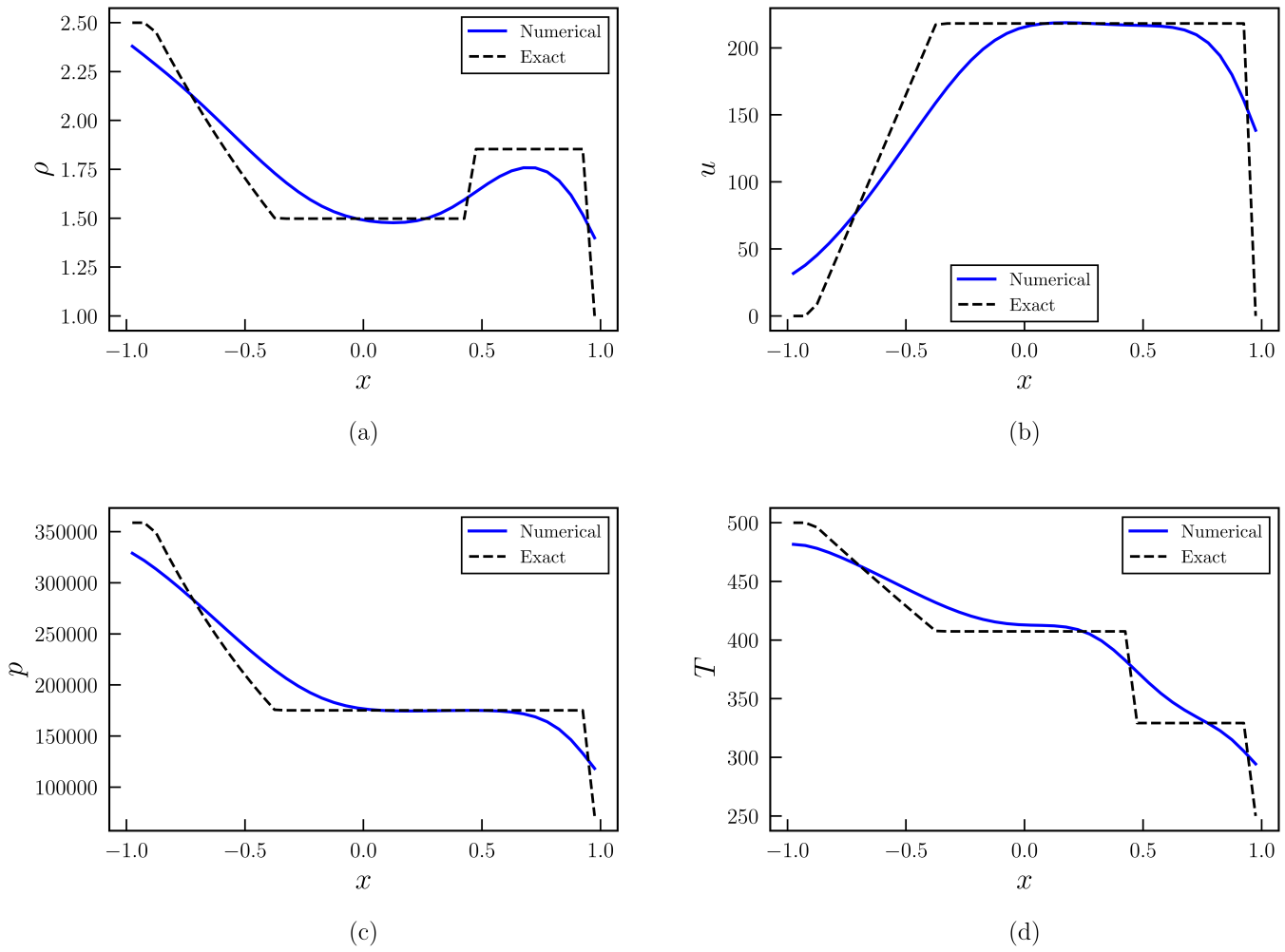


Figure 20: Comparison of numerical and exact solutions for Initial Condition 3 using the Steger–Warming flux scheme with RK4 time integration at $t = 0.0020$.

This scheme is unable to capture the sharp gradients, this could be attributed to the grid resolution. No effect of the timestepper is observed.

6.3.2 Van Leer (Local) Flux Scheme

6.3.2.1 Forward Euler Time Integration

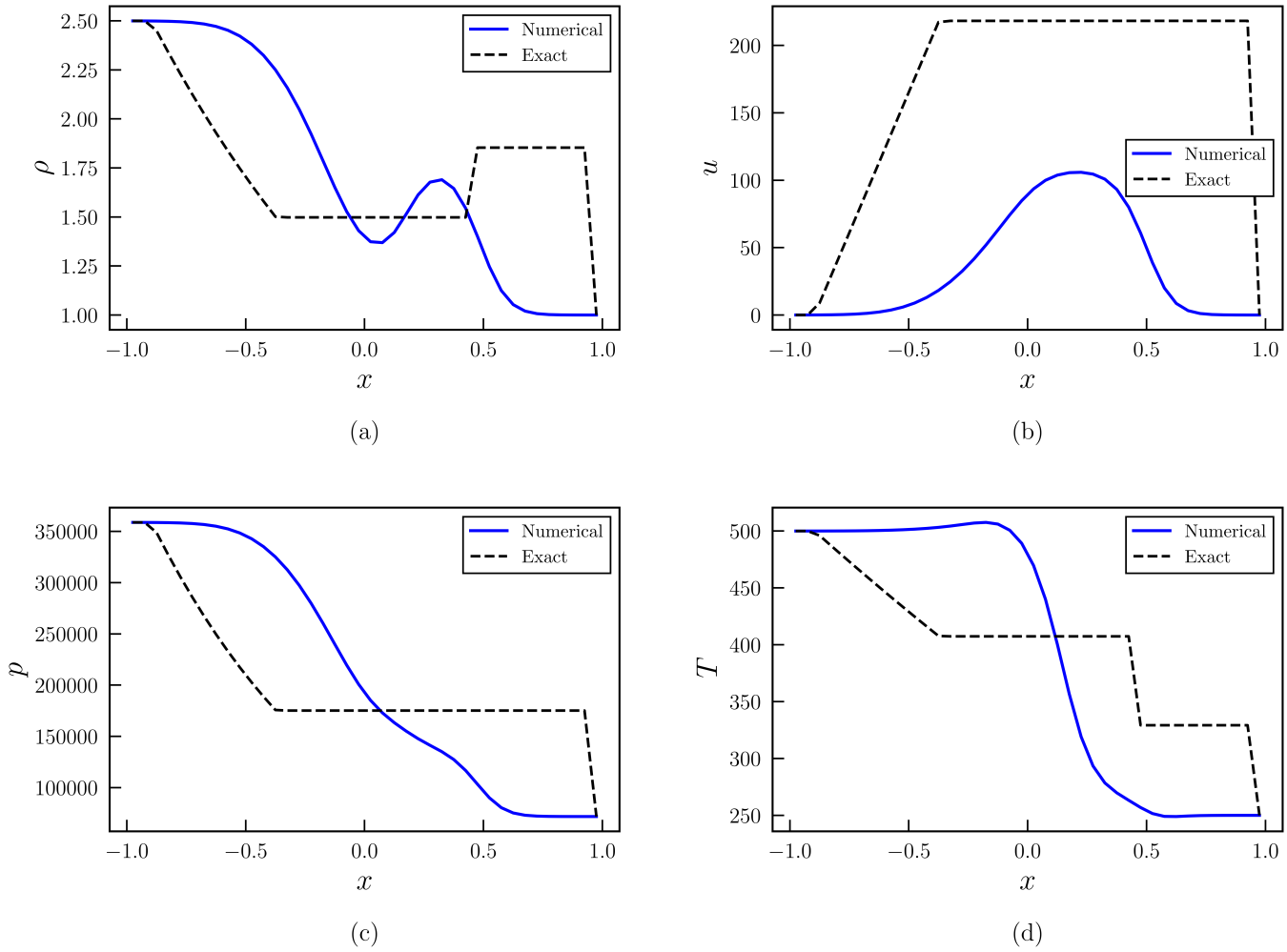


Figure 21: Comparison of numerical and exact solutions for Initial Condition 3 using the Van Leer (Local) flux scheme with Forward Euler time integration at $t = 0.0020$.

6.3.2.2 Runge-Kutta 4 Time Integration

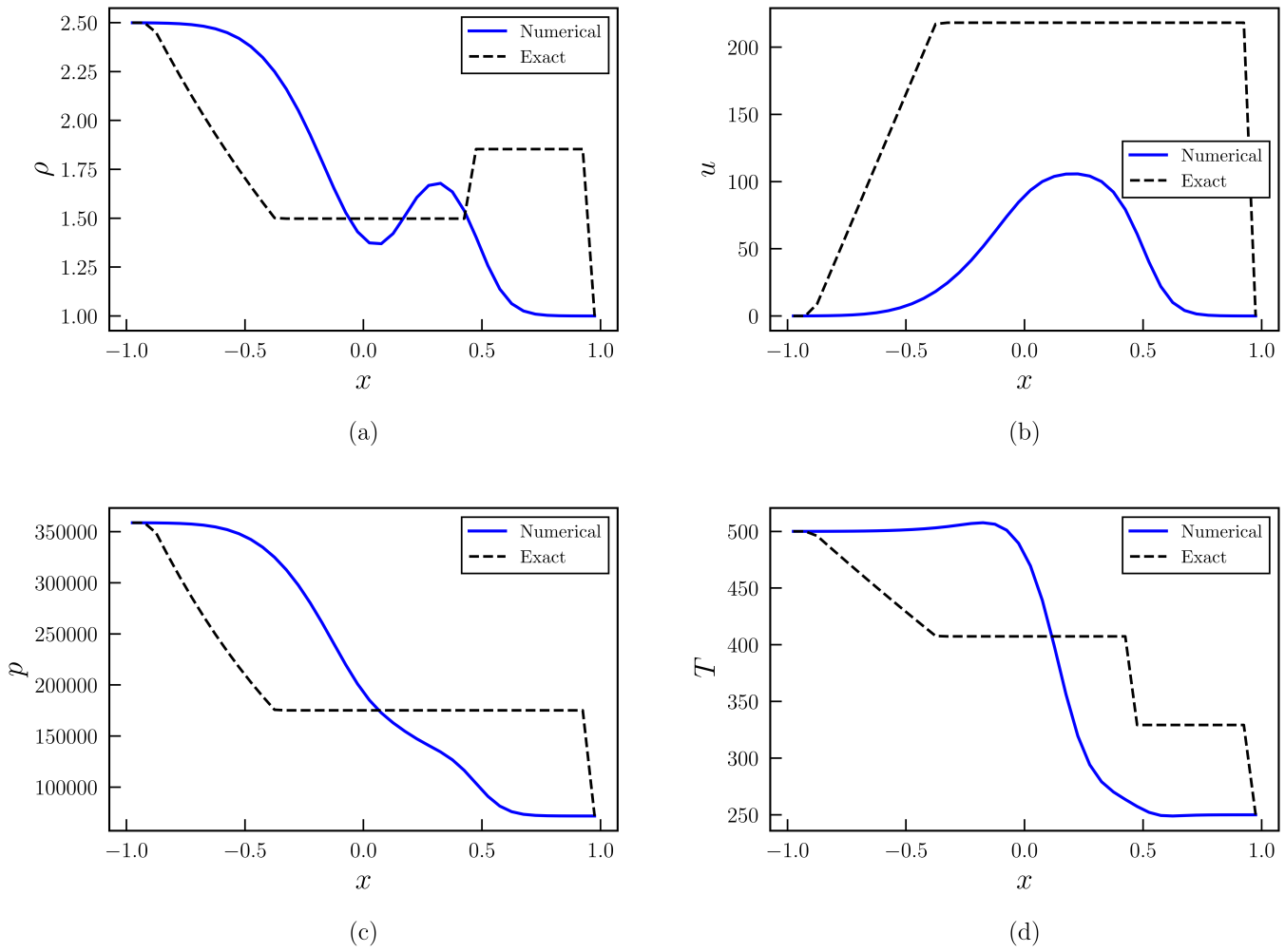


Figure 22: Comparison of numerical and exact solutions for Initial Condition 3 using the Van Leer (Local) flux scheme with RK4 time integration at $t = 0.0020$.

This scheme is also not able to capture the exact solution and no change is observed due to the timestepper.

6.3.3 Van Leer (Common) Flux Scheme

6.3.3.1 Forward Euler Time Integration

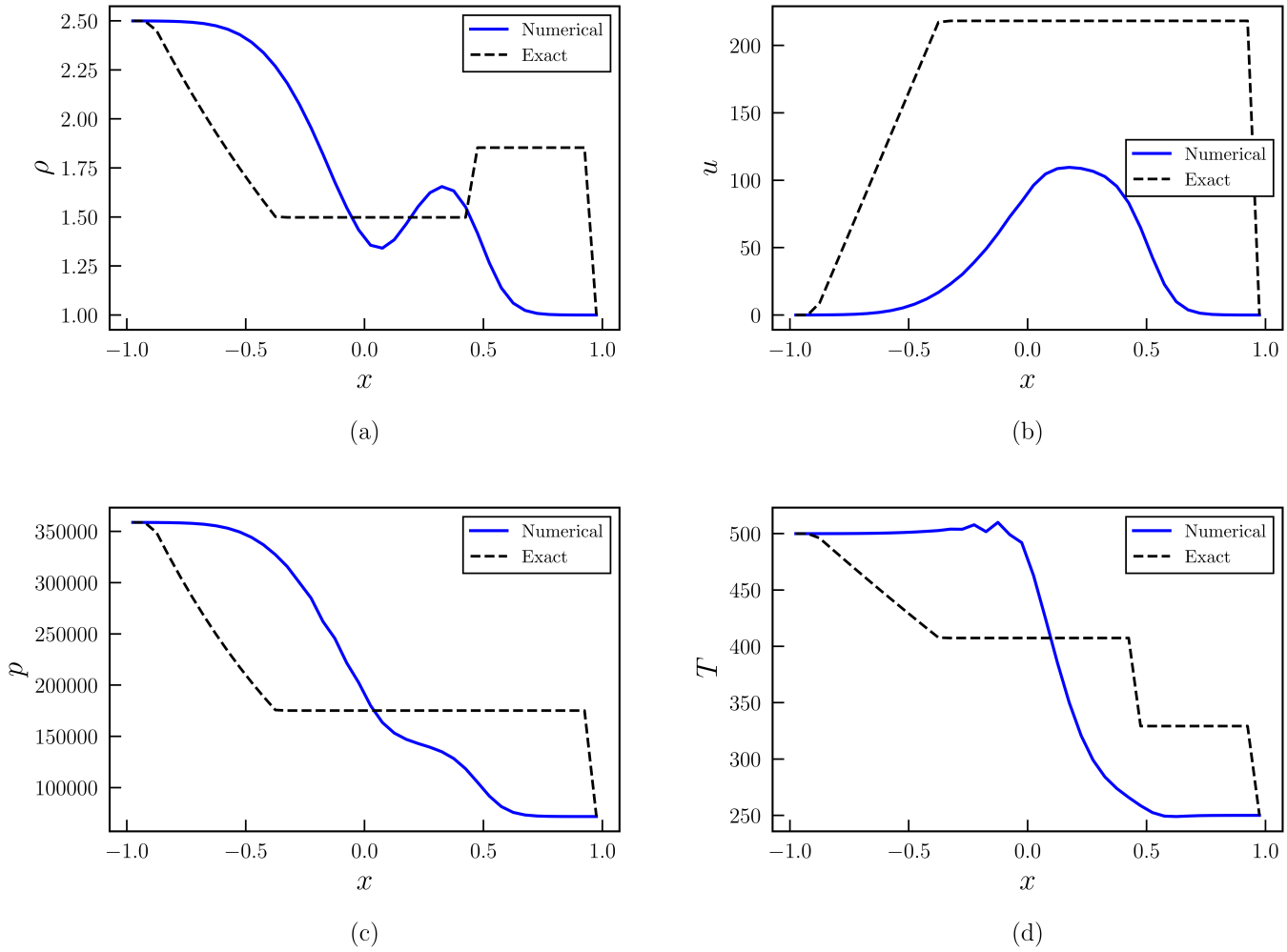


Figure 23: Comparison of numerical and exact solutions for Initial Condition 3 using the Van Leer (Common) flux scheme with Forward Euler time integration at $t = 0.0020$.

6.3.3.2 Runge-Kutta 4 Time Integration

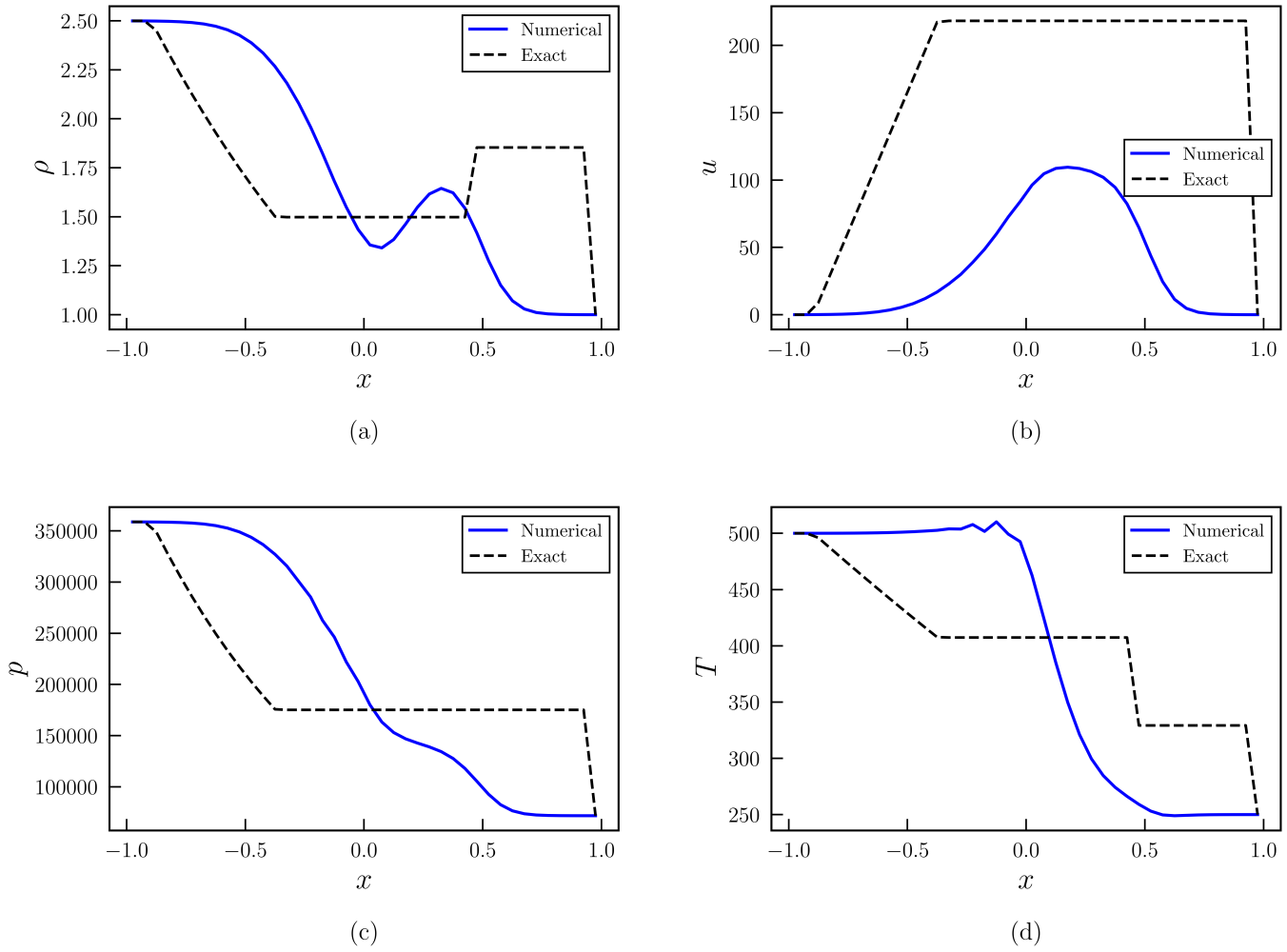


Figure 24: Comparison of numerical and exact solutions for Initial Condition 3 using the Van Leer (Common) flux scheme with RK4 time integration at $t = 0.0020$.

This scheme is also not able to capture the exact solution and no change is observed due to the timestepper.

6.3.4 AUSM Flux Scheme

6.3.4.1 Forward Euler Time Integration

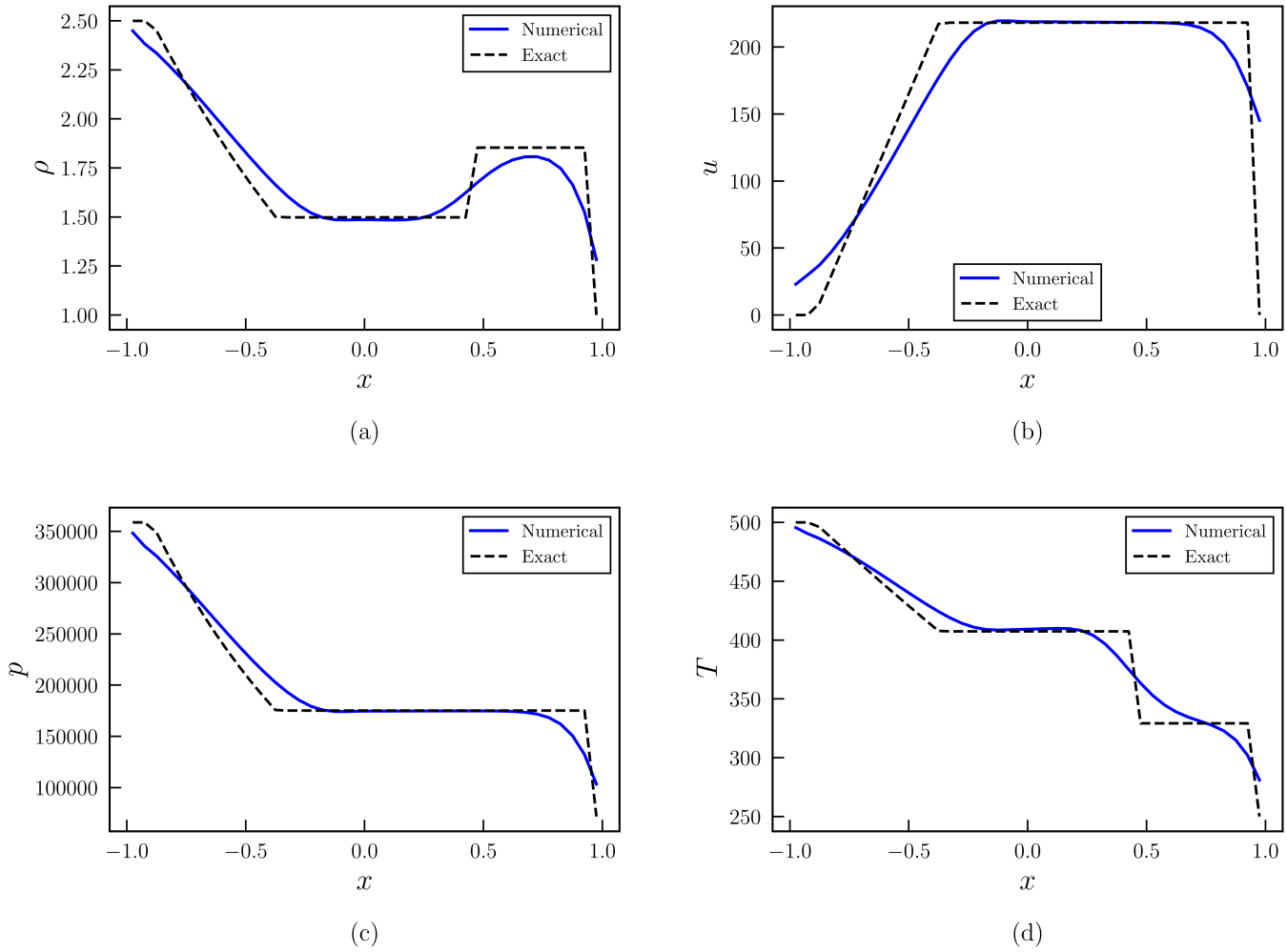


Figure 25: Comparison of numerical and exact solutions for Initial Condition 3 using the AUSM flux scheme with Forward Euler time integration at $t = 0.0020$.

6.3.4.2 Runge-Kutta 4 Time Integration

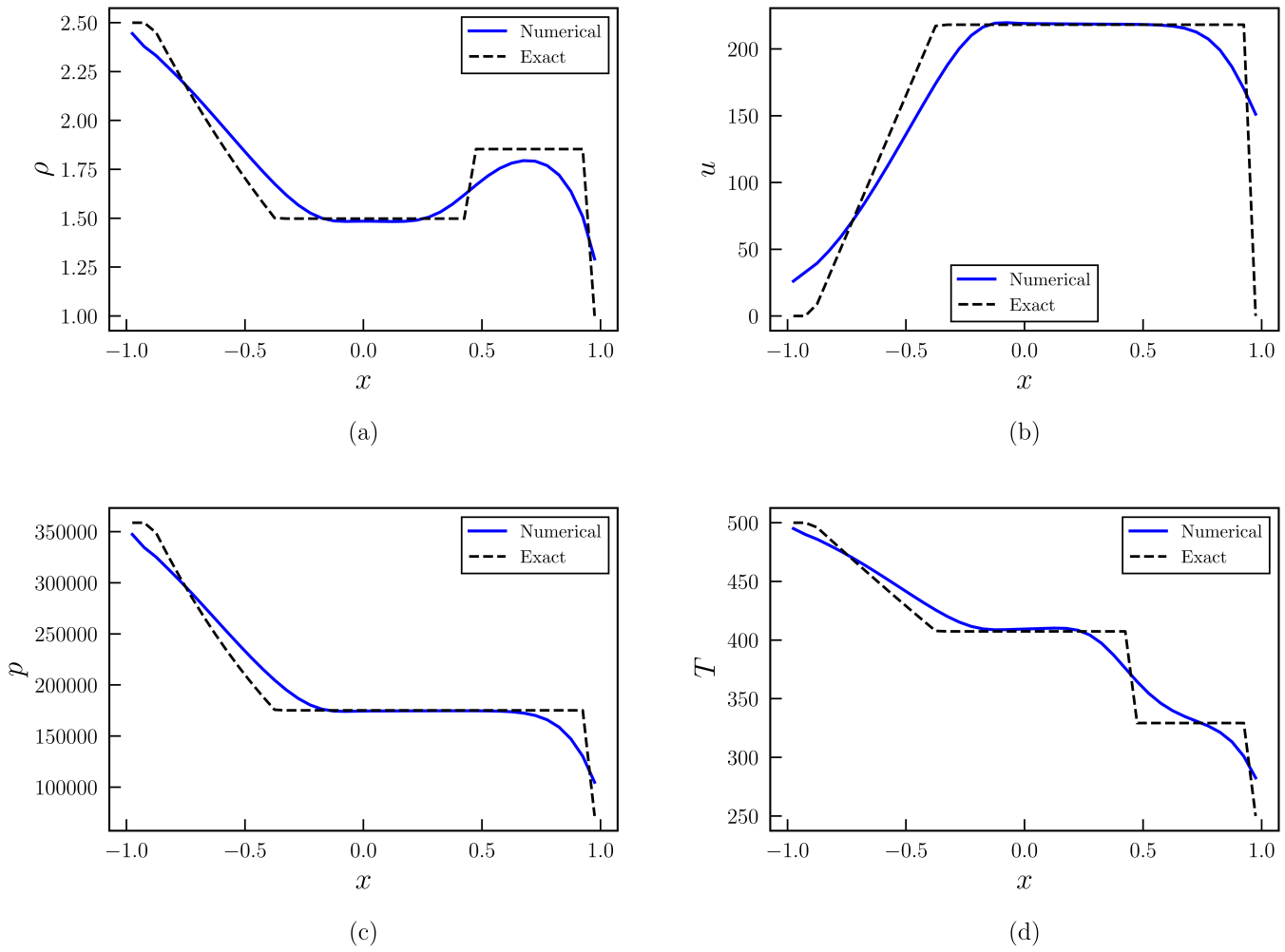


Figure 26: Comparison of numerical and exact solutions for Initial Condition 3 using the AUSM flux scheme with RK4 time integration at $t = 0.0020$.

The AUSM scheme is unable to capture the sharp gradients, this could be attributed to the grid resolution. No effect of the timestepper is observed.

6.3.5 Roe Flux Scheme

6.3.5.1 Forward Euler Time Integration

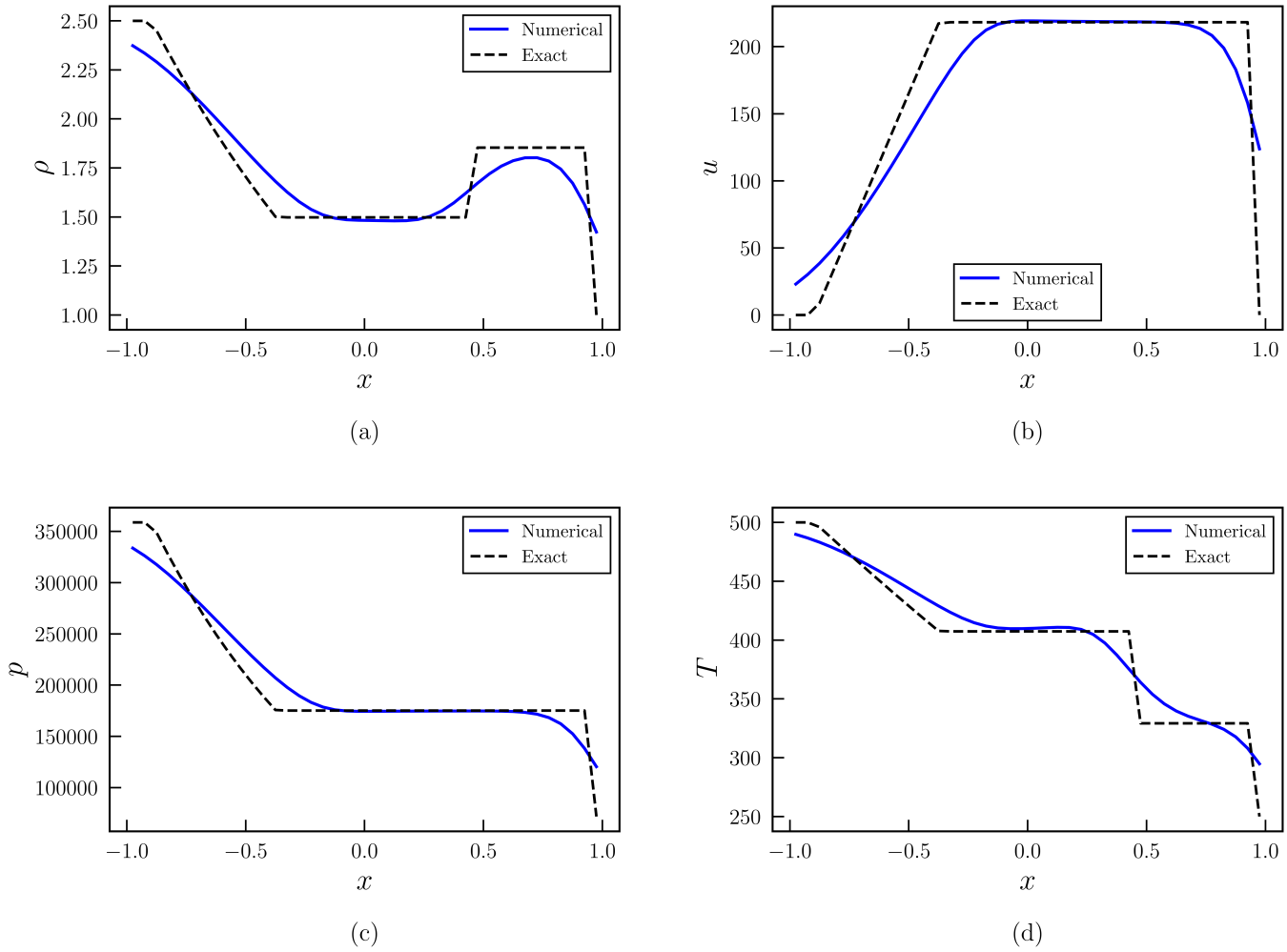


Figure 27: Comparison of numerical and exact solutions for Initial Condition 3 using the Roe flux scheme with Forward Euler time integration at $t = 0.0020$.

6.3.5.2 Runge-Kutta 4 Time Integration

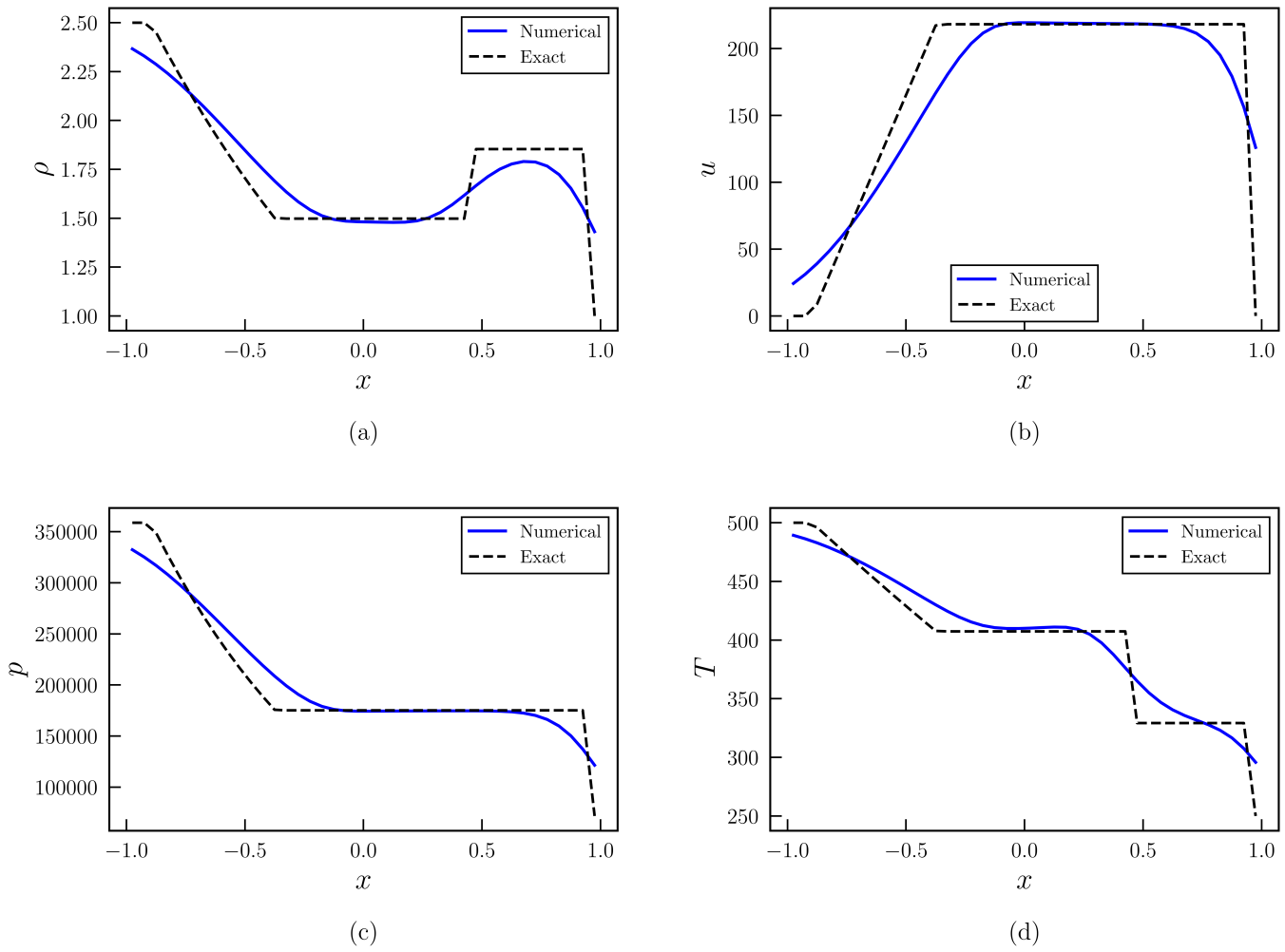


Figure 28: Comparison of numerical and exact solutions for Initial Condition 3 using the Roe flux scheme with RK4 time integration at $t = 0.0020$.

This scheme is unable to capture the sharp gradients, this could be attributed to the grid resolution. No effect of the timestepper is observed.

7 Conclusion

The inviscid Euler equation is solved numerically using the finite volume method. It is solved in an explicit fashion, with several flux construction schemes and initial conditions. The simulations were run in FORTRAN and post-processing is carried out using Python. The codes developed are exhibited in the appendix. Each of the schemes have their pitfalls. Grid resolution is also extremely critical as evidenced by the results presented in this report. It is therefore prudent to run the simulations using a scheme dependent on the physical scenario and at an appropriate resolution.

8 References

[1] LeVeque, R., *Finite Volume Methods for Hyperbolic Problems*, Cambridge Texts in Applied Mathematics, Cambridge University Press, 2002.

[2] Toro, E. F., *Riemann Solvers and Numerical Methods for Fluid Dynamics: A Practical Introduction*, 3rd ed., Springer, 2009.

A FORTRAN Implementation

A.1 Main Program

```
1  program euler
2
3      use var_dec
4      use sub
5      use exact
6      implicit none
7
8      integer :: nstep
9      integer :: output_count
10     character(len=100) :: filename
11     real(dp) :: QpL(4), QpR(4)
12     real(dp) :: x0, next_output_time
13     integer :: exact_unit
14     character(len=100) :: exact_filename
15     character(len=100) :: residual_filename
16     integer :: residual_unit
17     real(dp), parameter :: output_dt = 0.0002_dp
18     integer :: k
```

```
19
20      allocate(x(N+1), xc(N))
21      allocate(Qc(nvar,N+2), Qc_new(nvar,N+2))
22      allocate(Qp(nprim,N))
23      allocate(flux(nvar,N+1))
24      allocate(res(nvar,N))
25      allocate(Qp_exact(nprim,N))
26
27      dx = (xend - xstart) / real(N,dp)
28      do i = 1, N+1; x(i) = xstart + (i-1)*dx; end do
29      do i = 1, N; xc(i) = 0.5_dp*(x(i) + x(i+1)); end do
30
31      do icase = 1, 3
32
33          call ic(icase)
34          call get_riemann_states(icase, QpL, QpR)
35
36          x0 = 0.0_dp
37
38          write(exact_filename,'("exact-IC",I0,".dat")') icase
39          open(newunit=exact_unit, file=trim(exact_filename), status="replace", action="write")
40          write(exact_unit,'(A)') '# x rho u p T'
41
42          do k = 0, 10
43              call exact_riemann(xc, real(k,dp)*output_dt, x0, QpL, QpR, gamma, Qp_exact)
44              call write_exact_solution(exact_unit)
45          end do
46
47          close(exact_unit)
48
49          do fcase = 1, 5
50              do tcase = 1, 2
51
52                  case_failed = .false.
53
54                  call ic(icase)
55                  call prim_to_cons(Qp,Qc)
56
57                  t = 0.0_dp
58                  nstep = 0
59                  output_count = 0
```

```
60
61              write(filename,'("results-IC",I0,"-F",I0,"-T",I0,".dat")') icase, fcase, tcase
62              open(unit=20, file=trim(filename), status="replace", action="write")
63              write(20,'(A)') '# x rho u p T'
64
65              write(residual_filename,'("residual-IC",I0,"-F",I0,"-T",I0,".dat")') icase, fcase,
66              open(newunit=residual_unit, file=trim(residual_filename), status="replace", action=
67              write(residual_unit,'(A)') '# t max_rho max_momentum max_energy'
68
69              call cons_to_prim(Qc,Qp)
70              call write_solution(20)
71
72              next_output_time = 0.0002_dp
73
74              ! Time Stepping Loop
75              do while (t < tend)
76
77                  nstep = nstep + 1
78
79                  call compute_dt(Qc,dt)
80
81                  if (case_failed) then
82                      write(*,'(A,I0,A,I0,A,I0)') &
83                          'Skipping IC=',icase,' F=',fcase,' T=',tcase
84                      exit
85                  end if
86
87                  dt = min(dt, next_output_time - t)
88
89                  call compute_rhs(Qc,res)
90                  call write_residual(residual_unit, t, res)
91
92                  call time_integrator(Qc,Qc_new,dt,tcase)
93
94                  if (case_failed) then
95                      write(*,'(A,I0,A,I0,A,I0)') &
96                          'Skipping IC=',icase,' F=',fcase,' T=',tcase
97                      exit
98                  end if
99
100                 Qc = Qc_new
```

```
                    t = t + dt

                    call cons_to_prim(Qc,Qp)

                    if (t >= next_output_time - 1.0e-10_dp) then
                        call write_solution(20)
                        next_output_time = next_output_time + 0.0002_dp
                    end if

                end do

                close(20)
                close(residual_unit)
            end do
        end do
    end do

    deallocate(x,xc,Qc,Qc_new,Qp,Qp_exact,flux,res)

end program euler
```

A.2 Variable Declaration Module

```
module var_dec
    implicit none

    ! Define Double Precision
    integer, parameter :: dp = kind(1.d0)

    ! Number of Variables
    integer, parameter :: nvar = 3
    integer, parameter :: nprim = 4

    ! Initial Condition, Flux Scheme, Time Integrator
    integer :: icase, fcase, tcase

    ! Define Domain and Grid Parameters
    integer, parameter :: Lx = 2
    integer, parameter :: N = 40

    real(dp) :: xstart = -1.0_dp
    real(dp) :: xend = 1.0_dp
```

```
20
21      ! Define Gas Parameters
22      real(dp), parameter :: gamma = 1.4_dp
23      real(dp), parameter :: Rgas = 287.0_dp
24
25      ! Define Time Parameters
26      real(dp) :: dx, dt
27      real(dp) :: t, tend = 0.002_dp
28
29      ! CFL Number
30      real(dp), parameter :: CFL = 0.2_dp
31
32      ! Define Grid Arrays
33      real(dp), allocatable :: x(:), xc(:)
34
35      ! Conservative Variables
36      ! Qc(1,:) = Density
37      ! Qc(2,:) = Momentum
38      ! Qc(3,:) = Total Energy
39      real(dp), allocatable :: Qc(:,:), Qc_new(:,:)
40
41      ! Primitive Variables
42      ! Qp(1,:) = Density
43      ! Qp(2,:) = Velocity
44      ! Qp(3,:) = Pressure
45      ! Qp(4,:) = Temperature
46      real(dp), allocatable :: Qp(:,:)
47
48      ! Flux Arrays
49      real(dp), allocatable :: flux(:,:)
50
51      ! Residual Arrays
52      real(dp), allocatable :: res(:,:)
53
54      ! Output Arrays
55      real(dp), allocatable :: Qc_out_01(:,:), Qc_out_05(:,:)
56      real(dp), allocatable :: Qp_out_01(:,:), Qp_out_05(:,:)
57
58      ! Exact Solution Arrays
59      real(dp), allocatable :: Qp_exact(:,:)
60
```

```
61      ! Helper Variables
62      integer(kind=8) :: i, iface
63      logical :: case_failed
64
65  end module var_dec
```

A.3 Exact Solution Module

```
1   module exact
2       use var_dec, only: dp, Rgas
3       implicit none
4       private
5       public :: exact_riemann
6
7   contains
8
9       pure real(dp) function pf(p,rho,pk,a,g) result(f)
10          real(dp),intent(in)::p,rho,pk,a,g
11          real(dp)::coef,B
12
13          if(p>pk)then
14              coef=2.0_dp/((g+1.0_dp)*rho); B=(g-1.0_dp)*pk/(g+1.0_dp)
15              f=(p-pk)*sqrt(coef/(p+B))
16          else
17              f=2.0_dp*a/(g-1.0_dp)*((p/pk)**((g-1.0_dp)/(2.0_dp*g))-1.0_dp)
18          endif
19      end function
20
21      subroutine starp(rL,uL,pL,aL,rR,uR,pR,aR,g,ps)
22          real(dp),intent(in)::rL,uL,pL,aL,rR,uR,pR,aR,g
23          real(dp),intent(out)::ps
24          real(dp)::lo,hi,mid,f
25          integer::i
26
27          lo=0.0_dp; hi=max(pL,pR)
28
29          do while(pf(hi,rL,pL,aL,g)+pf(hi,rR,pR,aR,g)+uR-uL<0.0_dp)
30              hi=2.0_dp*hi
31          enddo
32
33          do i=1,100
34              mid=0.5_dp*(lo+hi)
```

```
35          f=pf(mid,rL,pL,aL,g)+pf(mid,rR,pR,aR,g)+uR-uL
36          if(abs(f)<1.0e-12_dp)exit
37          if(f>0.0_dp)then
38              hi=mid
39          else
40              lo=mid
41          endif
42      enddo
43
44      ps=mid
45  end subroutine
46
47  subroutine exact_riemann(x,t,x0,QL,QR,g,Q)
48      real(dp),intent(in)::x(:),t,x0,QL(4),QR(4),g
49      real(dp),intent(out)::Q(:,:)
50      real(dp)::rL,uL,pL,aL,rR,uR,pR,aR,ps,us,rsL,rsR
51      real(dp)::asL,asR,SL,SR,SHL,STL,SHR,STR,xi,r,u,p,a,rg
52      integer::i
53
54      rL=QL(1); uL=QL(2); pL=QL(3); aL=sqrt(g*pL/rL)
55      rR=QR(1); uR=QR(2); pR=QR(3); aR=sqrt(g*pR/rR)
56      rg=Rgas
57      if(t<=0.0_dp)then
58          do i=1,size(x)
59              Q(:,i)=merge(QL,QR,x(i)<x0)
60          enddo
61          return
62      endif
63      call starp(rL,uL,pL,aL,rR,uR,pR,aR,g,ps)
64      us=0.5_dp*(uL+uR+pf(ps,rR,pR,aR,g)-pf(ps,rL,pL,aL,g))
65      if(ps>pL)then
66          rsL=rL*(ps/pL+(g-1.0_dp)/(g+1.0_dp))/ &
67              ((g-1.0_dp)/(g+1.0_dp)*ps/pL+1.0_dp)
68          SL=uL-aL*sqrt((g+1.0_dp)*ps/(2.0_dp*g*pL)+(g-1.0_dp)/(2.0_dp*g))
69      else
70          rsL=rL*(ps/pL)**(1.0_dp/g); asL=aL*(ps/pL)**((g-1.0_dp)/(2.0_dp*g))
71          SHL=uL-aL; STL=us-asL
72      endif
73      if(ps>pR)then
74          rsR=rR*(ps/pR+(g-1.0_dp)/(g+1.0_dp))/ &
75              ((g-1.0_dp)/(g+1.0_dp)*ps/pR+1.0_dp)
```

```
76              SR=uR+aR*sqrt((g+1.0_dp)*ps/(2.0_dp*g*pR)+(g-1.0_dp)/(2.0_dp*g))
77          else
78              rsR=rR*(ps/pR)**(1.0_dp/g); asR=aR*(ps/pR)**((g-1.0_dp)/(2.0_dp*g))
79              SHR=uR+aR; STR=us+asR
80          endif
81          do i=1,size(x)
82              xi=(x(i)-x0)/t
83              if(xi<=us)then
84                  if(ps>pL)then
85                      if(xi<=SL)then
86                          r=rL; u=uL; p=pL
87                      else
88                          r=rsL; u=us; p=ps
89                      endif
90                  else
91                      if(xi<=SHL)then
92                          r=rL; u=uL; p=pL
93                      else if(xi>=STL)then
94                          r=rsL; u=us; p=ps
95                      else
96                          u=2.0_dp/(g+1.0_dp)*(aL+0.5_dp*(g-1.0_dp)*uL+xi)
97                          a=2.0_dp/(g+1.0_dp)*(aL+0.5_dp*(g-1.0_dp)*(uL-xi))
98                          r=rL*(a/aL)**(2.0_dp/(g-1.0_dp))
99                          p=pL*(a/aL)**(2.0_dp*g/(g-1.0_dp))
100                     endif
101                 endif
102             else
103                 if(ps>pR)then
104                     if(xi>=SR)then
105                         r=rR; u=uR; p=pR
106                     else
107                         r=rsR; u=us; p=ps
108                     endif
109                 else
110                     if(xi>=SHR)then
111                         r=rR; u=uR; p=pR
112                     else if(xi<=STR)then
113                         r=rsR; u=us; p=ps
114                     else
115                         u=2.0_dp/(g+1.0_dp)*(-aR+0.5_dp*(g-1.0_dp)*uR+xi)
116                         a=2.0_dp/(g+1.0_dp)*(aR-0.5_dp*(g-1.0_dp)*(uR-xi))
```

```
                    r=rR*(a/aR)**(2.0_dp/(g-1.0_dp))
                    p=pR*(a/aR)**(2.0_dp*g/(g-1.0_dp))
                endif
            endif
        endif
        Q(:,i)=[r,u,p,p/(r*rg)]
    enddo
  end subroutine
end module exact
```

A.4 Subroutine Module

```
module sub
  use var_dec
  implicit none

contains

  subroutine ic(which)
    integer, intent(in) :: which
    real(dp) :: rhoL,uL,pL,TL,rhoR,uR,pR,TR,M,aL,pRatio,rhoRatio
    integer :: j

    Qp=0.0_dp
    select case(which)
    case(1)
      rhoL=1.0_dp; uL=0.0_dp; TL=500.0_dp
      rhoR=2.5_dp; uR=0.0_dp; TR=200.0_dp
      pL=rhoL*Rgas*TL; pR=rhoR*Rgas*TR
    case(2)
      pL=100.0_dp; TL=401.8_dp; M=3.0_dp
      rhoL=pL/(Rgas*TL); aL=sqrt(gamma*Rgas*TL); uL=M*aL
      pRatio=1.0_dp+2.0_dp*gamma/(gamma+1.0_dp)*(M*M-1.0_dp)
      rhoRatio=(gamma+1.0_dp)*M*M/((gamma-1.0_dp)*M*M+2.0_dp)
      pR=pL*pRatio; rhoR=rhoL*rhoRatio
      TR=TL*pRatio/rhoRatio; uR=uL/rhoRatio
    case(3)
      rhoL=2.5_dp; uL=0.0_dp; TL=500.0_dp
      rhoR=1.0_dp; uR=0.0_dp; TR=250.0_dp
      pL=rhoL*Rgas*TL; pR=rhoR*Rgas*TR
    case default
      error stop "ic: invalid case"
```

```
        end select
        do j=1,N
            if (xc(j)<=0.0_dp) then
                Qp(:,j)=[rhoL, uL, pL, TL]
            else
                Qp(:,j)=[rhoR, uR, pR, TR]
            end if
        end do
    end subroutine ic

    subroutine prim_to_cons(Qp,Qc)
        real(dp),intent(in)::Qp(:,:)
        real(dp),intent(out)::Qc(:,:)
        integer::j

        Qc=0.0_dp
        do j=1,N
            Qc(:,j+1)=primitive_to_conservative(Qp(:,j))
        end do
    end subroutine prim_to_cons

    function primitive_to_conservative(Qp) result(Qc)
        real(dp),intent(in)::Qp(4)
        real(dp)::Qc(3),rho,u,p,T

        rho=Qp(1);u=Qp(2);p=Qp(3);T=Qp(4)
        Qc=[rho,rho*u,p/(gamma-1.0_dp)+0.5_dp*rho*u*u]
    end function primitive_to_conservative

    subroutine cons_to_prim(Qc,Qp)
        real(dp),intent(in)::Qc(:,:)
        real(dp),intent(out)::Qp(:,:)
        integer::j
        real(dp)::rho,u,p

        do j=1,N
            rho=Qc(1,j+1)
            if (rho<=1e-12_dp) error stop "cons_to_prim: non-positive density"
            u=Qc(2,j+1)/rho
            p=(gamma-1.0_dp)*(Qc(3,j+1)-0.5_dp*Qc(2,j+1)*u)
            if (p<=1.0e-10_dp) then
```

```
72              write(*,'(A,I0,A,ES14.6,A,ES14.6)') "bad cell ",j," rho=",rho," p=",p
73              case_failed=.true.
74              return
75          end if
76          Qp(:,j)=[rho, u, p, p/(rho*Rgas)]
77      end do
78  end subroutine
79
80  subroutine boundary_conditions(Qc)
81      real(dp),intent(inout)::Qc(:,:)
82
83      ! Transmissive (zero-gradient) boundary conditions for ghost cells
84      Qc(:,1)=Qc(:,2)
85      Qc(:,N+2)=Qc(:,N+1)
86  end subroutine
87
88  subroutine compute_dt(Qc,dtloc)
89      real(dp),intent(in)::Qc(:,:)
90      real(dp),intent(out)::dtloc
91      integer::j
92      real(dp)::rho,u,p,a,smax
93
94      smax=0.0_dp
95      do j=2,N+1
96          rho=Qc(1,j);u=Qc(2,j)/rho
97          p=(gamma-1.0_dp)*(Qc(3,j)-0.5_dp*Qc(2,j)*u)
98          if (rho <= 1.0e-12_dp .or. p <= 1.0e-10_dp) then
99              case_failed = .true.
100             dtloc = 0.0_dp
101             return
102         end if
103         a=sqrt(gamma*p/rho)
104         smax=max(smax,abs(u)+a)
105     end do
106     dtloc=CFL*dx/smax
107 end subroutine
108
109 subroutine compute_rhs(Qc,rhs)
110     real(dp),intent(in)::Qc(:,:)
111     real(dp),intent(out)::rhs(:,:)
112     real(dp)::Ub(3,N+2),F(3,N+1)
```

```
113         integer::j
114
115         Ub=Qc
116         call boundary_conditions(Ub)
117         do j=1,N+1
118             F(:,j)=numerical_flux(Ub(:,j),Ub(:,j+1),fcase)
119             if(case_failed) then
120                 rhs=0.0_dp
121                 return
122             end if
123         end do
124         do j=1,N
125             rhs(:,j)=-(F(:,j+1)-F(:,j))/dx
126         end do
127     end subroutine
128
129     function numerical_flux(QcLin,QcRin,scheme) result(Fout)
130         real(dp),intent(in)::QcLin(3),QcRin(3)
131         integer,intent(in)::scheme
132         real(dp)::Fout(3),Qpleft(4),Qpright(4)
133         real(dp)::HL,HR,rhol,uloc,pl,rhor,uright,pr,al,ar
134
135         Qpleft=to_primitive(QcLin);Qpright=to_primitive(QcRin)
136         if (case_failed) then
137             Fout = 0.0_dp
138             return
139         end if
140         rhol=Qpleft(1);uloc=Qpleft(2);pl=Qpleft(3)
141         rhor=Qpright(1);uright=Qpright(2);pr=Qpright(3)
142         al=sqrt(gamma*pl/rhol);ar=sqrt(gamma*pr/rhor)
143         HL=(QcLin(3)+pl)/rhol;HR=(QcRin(3)+pr)/rhor
144
145         select case(scheme)
146             case(1)
147                 Fout=sw_flux(QcLin,QcRin)
148             case(2)
149                 Fout=vanleer_flux(Qpleft,Qpright,al,ar,.false.)
150             case(3)
151                 Fout=vanleer_flux(Qpleft,Qpright,al,ar,.true.)
152             case(4)
153                 Fout=ausm_flux(Qpleft,Qpright,al,ar)
```

```
154             case(5)
155                 Fout=roe_flux(QcLin,QcRin,Qpleft,Qpright,HL,HR)
156         end select
157     end function
158 
159     function to_primitive(Qc) result(Qp)
160         real(dp),intent(in)::Qc(3)
161         real(dp)::Qp(4),rho,u,p
162 
163         rho=Qc(1)
164 
165         if(rho<=1.0e-12_dp) then
166             case_failed=.true.
167             Qp=0.0_dp
168             return
169         end if
170 
171         u=Qc(2)/rho
172         p=(gamma-1.0_dp)*(Qc(3)-0.5_dp*Qc(2)*u)
173 
174         if(p<=1.0e-10_dp) then
175             case_failed=.true.
176             Qp=0.0_dp
177             return
178         end if
179 
180         Qp=[rho, u, p, p/(rho*Rgas)]
181     end function
182 
183     function sw_flux(QcL,QcR) result(F)
184         real(dp),intent(in)::QcL(3),QcR(3)
185         real(dp)::F(3),QpL(4),QpR(4),FLp(3),FLm(3),FRp(3),FRm(3)
186 
187         QpL=to_primitive(QcL);QpR=to_primitive(QcR)
188         FLp=sw_split(QcL,QpL,.true.);FLm=sw_split(QcL,QpL,.false.)
189         FRp=sw_split(QcR,QpR,.true.);FRm=sw_split(QcR,QpR,.false.)
190         F=FLp+FRm
191     end function
192 
193     function sw_split(Qc,Qp,positive) result(Fs)
194         real(dp),intent(in)::Qc(3),Qp(3)
```

```
195         logical,intent(in)::positive
196         real(dp)::Fs(3),rho,u,p,a,H,lam(3),Rmat(3,3),Lmat(3,3),coef(3),lp(3)
197         integer::k
198
199         rho=Qp(1);u=Qp(2);p=Qp(3);a=sqrt(gamma*p/rho)
200         H=(Qc(3)+p)/rho
201
202         Rmat(:,1)=[1.0_dp,u,0.5_dp*u*u]
203         Rmat(:,2)=[1.0_dp,u+a,H+u*a]
204         Rmat(:,3)=[1.0_dp,u-a,H-u*a]
205
206         call invert3(Rmat,Lmat)
207         coef=matmul(Lmat,Qc)
208         lam=[u,u+a,u-a]
209
210         do k=1,3
211             if(positive) then
212                 lp(k)=0.5_dp*(lam(k)+abs(lam(k)))
213             else
214                 lp(k)=0.5_dp*(lam(k)-abs(lam(k)))
215             end if
216         end do
217
218         Fs=matmul(Rmat,lp*coef)
219     end function
220
221     subroutine invert3(A,B)
222         real(dp),intent(in)::A(3,3)
223         real(dp),intent(out)::B(3,3)
224         real(dp)::det
225
226         det=A(1,1)*(A(2,2)*A(3,3)-A(2,3)*A(3,2)) &
227            -A(1,2)*(A(2,1)*A(3,3)-A(2,3)*A(3,1)) &
228            +A(1,3)*(A(2,1)*A(3,2)-A(2,2)*A(3,1))
229
230         if(abs(det)<1.0e-14_dp) error stop "invert3: singular matrix"
231
232         B(1,1)=(A(2,2)*A(3,3)-A(2,3)*A(3,2))/det
233         B(1,2)=(A(1,3)*A(3,2)-A(1,2)*A(3,3))/det
234         B(1,3)=(A(1,2)*A(2,3)-A(1,3)*A(2,2))/det
235         B(2,1)=(A(2,3)*A(3,1)-A(2,1)*A(3,3))/det
```

```
236         B(2,2)=(A(1,1)*A(3,3)-A(1,3)*A(3,1))/det
237         B(2,3)=(A(1,3)*A(2,1)-A(1,1)*A(2,3))/det
238         B(3,1)=(A(2,1)*A(3,2)-A(2,2)*A(3,1))/det
239         B(3,2)=(A(1,2)*A(3,1)-A(1,1)*A(3,2))/det
240         B(3,3)=(A(1,1)*A(2,2)-A(1,2)*A(2,1))/det
241     end subroutine
242
243     function vanleer_flux(Qpleft,Qpright,aL,aR,common) result(Fout)
244         real(dp),intent(in)::Qpleft(4),Qpright(4),aL,aR
245         logical,intent(in)::common
246         real(dp)::Fout(3),alocal,arocal,ML,MR
247
248         if(common) then
249             alocal=max(aL,aR);arocal=alocal
250         else
251             alocal=aL;arocal=aR
252         end if
253
254         ML=Qpleft(2)/alocal;MR=Qpright(2)/arocal
255         Fout=vl_split(Qpleft,alocal,ML,.true.)+vl_split(Qpright,arocal,MR,.false.)
256     end function
257
258     function vl_split(Qp,a,M,plus) result(Fp)
259         real(dp),intent(in)::Qp(4),a,M
260         logical,intent(in)::plus
261         real(dp)::Fp(3),rho,u,p,fac
262
263         rho=Qp(1);u=Qp(2);p=Qp(3)
264
265         if(plus) then
266             if(M>=1.0_dp) then
267                 Fp=[rho*u,rho*u*u+p,u*(rho*(0.5_dp*u*u)+gamma*p/(gamma-1.0_dp))]
268             else if(M<=-1.0_dp) then
269                 Fp=0.0_dp
270             else
271                 fac=0.25_dp*rho*a*(M+1.0_dp)**2
272                 Fp=fac*[1.0_dp, 2.0_dp*a/gamma*(M+(gamma-1.0_dp)/2.0_dp), &
273                     2.0_dp*a*a/(gamma*gamma-1.0_dp)*(0.5_dp*(gamma-1.0_dp)*M*M+M+1.0_dp/(gamma+1.0_
274             end if
275         else
276             if(M<=-1.0_dp) then
```

```
277             Fp=[rho*u,rho*u*u+p,u*(rho*(0.5_dp*u*u)+gamma*p/(gamma-1.0_dp))]
278         else if(M>=1.0_dp) then
279             Fp=0.0_dp
280         else
281             fac=-0.25_dp*rho*a*(M-1.0_dp)**2
282             Fp=fac*[1.0_dp, 2.0_dp*a/gamma*(M-(gamma-1.0_dp)/2.0_dp), &
283                 2.0_dp*a*a/(gamma*gamma-1.0_dp)*(0.5_dp*(gamma-1.0_dp)*M*M-M+1.0_dp/(gamma+1.0_
284         end if
285     end if
286 end function
287
288 function ausm_flux(Qpleft,Qpright,aL,aR) result(Fout)
289     real(dp),intent(in)::Qpleft(4),Qpright(4),aL,aR
290     real(dp)::Fout(3)
291     real(dp)::rhoL,uL,pL,HL, rhoR,uR,pR,HR
292     real(dp)::a_half,ML,MR,Mhalf,phalf
293     real(dp)::massflux,uface,Hface
294
295     rhoL=Qpleft(1);uL=Qpleft(2);pL=Qpleft(3)
296     rhoR=Qpright(1);uR=Qpright(2);pR=Qpright(3)
297
298     HL=0.5_dp*uL*uL + gamma/(gamma-1.0_dp)*pL/rhoL
299     HR=0.5_dp*uR*uR + gamma/(gamma-1.0_dp)*pR/rhoR
300
301     a_half=max(aL,aR)
302     ML=uL/a_half; MR=uR/a_half
303
304     Mhalf=mach_plus(ML)+mach_minus(MR)
305     phalf=pressure_plus(ML)*pL + pressure_minus(MR)*pR
306
307     if(Mhalf>=0.0_dp) then
308         massflux=a_half*Mhalf*rhoL
309         uface=uL; Hface=HL
310     else
311         massflux=a_half*Mhalf*rhoR
312         uface=uR; Hface=HR
313     end if
314
315     Fout(1)=massflux; Fout(2)=massflux*uface+phalf; Fout(3)=massflux*Hface
316 end function ausm_flux
317
```

```
318     function mach_plus(M) result(v)
319         real(dp),intent(in)::M
320         real(dp)::v
321 
322         if(abs(M)>=1.0_dp) then
323             v=0.5_dp*(M+abs(M))
324         else
325             v=0.25_dp*(M+1.0_dp)**2
326         end if
327     end function
328 
329     function mach_minus(M) result(v)
330         real(dp),intent(in)::M
331         real(dp)::v
332 
333         if(abs(M)>=1.0_dp) then
334             v=0.5_dp*(M-abs(M))
335         else
336             v=-0.25_dp*(M-1.0_dp)**2
337         end if
338     end function
339 
340     function pressure_plus(M) result(v)
341         real(dp),intent(in)::M
342         real(dp)::v
343 
344         if(abs(M)>=1.0_dp) then
345             v=0.5_dp*(1.0_dp+sign(1.0_dp,M))
346         else
347             v=0.25_dp*(M+1.0_dp)**2*(2.0_dp-M)
348         end if
349     end function
350 
351     function pressure_minus(M) result(v)
352         real(dp),intent(in)::M
353         real(dp)::v
354 
355         if(abs(M)>=1.0_dp) then
356             v=0.5_dp*(1.0_dp-sign(1.0_dp,M))
357         else
358             v=0.25_dp*(M-1.0_dp)**2*(2.0_dp+M)
```

```
359         end if
360     end function
361
362     function roe_flux(QcL,QcR,QpL,QpR,HL,HR) result(F)
363         real(dp),intent(in)::QcL(3),QcR(3),QpL(4),QpR(4),HL,HR
364         real(dp)::F(3),ubar,Hbar,rbar,abar,Rmat(3,3),alpha(3),lam(3)
365         real(dp)::s1,s2,s3
366         real(dp)::drho,duvel,dpres
367
368         rbar=sqrt(QpL(1)*QpR(1))
369         ubar=(sqrt(QpL(1))*QpL(2)+sqrt(QpR(1))*QpR(2))/(sqrt(QpL(1))+sqrt(QpR(1)))
370         Hbar=(sqrt(QpL(1))*HL+sqrt(QpR(1))*HR)/(sqrt(QpL(1))+sqrt(QpR(1)))
371         abar=sqrt(max(0.0_dp,(gamma-1.0_dp)*(Hbar-0.5_dp*ubar*ubar)))
372
373         Rmat(:,1)=[1.0_dp,ubar,0.5_dp*ubar*ubar]
374         Rmat(:,2)=[1.0_dp,ubar+abar,Hbar+ubar*abar]
375         Rmat(:,3)=[1.0_dp,ubar-abar,Hbar-ubar*abar]
376
377         drho = QpR(1)-QpL(1)
378         duvel = QpR(2)-QpL(2)
379         dpres = QpR(3)-QpL(3)
380
381         alpha(1)=drho-dpres/(abar*abar)
382         alpha(2)=(dpres+rbar*abar*duvel)/(2.0_dp*abar*abar)
383         alpha(3)=(dpres-rbar*abar*duvel)/(2.0_dp*abar*abar)
384
385         lam=[ubar,ubar+abar,ubar-abar]
386         s1=abs(lam(1));s2=abs(lam(2));s3=abs(lam(3))
387         F=0.5_dp*(physical_flux(QcL)+physical_flux(QcR)) &
388           -0.5_dp*(Rmat(:,1)*s1*alpha(1)+Rmat(:,2)*s2*alpha(2)+Rmat(:,3)*s3*alpha(3))
389     end function
390
391     function physical_flux(Qc) result(f)
392         real(dp),intent(in)::Qc(3)
393         real(dp)::f(3),rho,u,p
394
395         rho=Qc(1);u=Qc(2)/rho
396         p=(gamma-1.0_dp)*(Qc(3)-0.5_dp*Qc(2)*u)
397         f=[Qc(2),Qc(2)*u+p,u*(Qc(3)+p)]
398     end function
399
```

```
400     subroutine time_integrator(Qc,Qcout,dtloc,which)
401         real(dp),intent(in)::Qc(:,:),dtloc
402         real(dp),intent(out)::Qcout(:,:)
403         integer,intent(in)::which
404         real(dp)::k1(3,N),k2(3,N),k3(3,N),k4(3,N),Us(3,N+2)
405 
406         select case(which)
407             case(1)
408                 call compute_rhs(Qc,k1)
409                 if(case_failed) then
410                     Qcout=Qc
411                     return
412                 end if
413                 Qcout=Qc
414                 Qcout(:,2:N+1)=Qc(:,2:N+1)+dtloc*k1
415                 call boundary_conditions(Qcout)
416             case(2)
417                 call compute_rhs(Qc,k1)
418                 if(case_failed) then
419                     Qcout=Qc
420                     return
421                 end if
422                 Us=Qc;Us(:,2:N+1)=Qc(:,2:N+1)+0.5_dp*dtloc*k1
423                 call boundary_conditions(Us)
424                 call compute_rhs(Us,k2)
425                 if(case_failed) then
426                     Qcout=Qc
427                     return
428                 end if
429                 Us=Qc;Us(:,2:N+1)=Qc(:,2:N+1)+0.5_dp*dtloc*k2
430                 call boundary_conditions(Us)
431                 call compute_rhs(Us,k3)
432                 if(case_failed) then
433                     Qcout=Qc
434                     return
435                 end if
436                 Us=Qc;Us(:,2:N+1)=Qc(:,2:N+1)+dtloc*k3
437                 call boundary_conditions(Us)
438                 call compute_rhs(Us,k4)
439                 if(case_failed) then
440                     Qcout=Qc
```

```
441                 return
442             end if
443             Qcout=Qc
444             Qcout(:,2:N+1)=Qc(:,2:N+1)+dtloc/6.0_dp*(k1+2.0_dp*k2+2.0_dp*k3+k4)
445             call boundary_conditions(Qcout)
446         end select
447     end subroutine
448
449     subroutine write_solution(unitno)
450         integer,intent(in)::unitno
451         integer::j
452         real(dp)::rho,u,p,T
453
454         do j=1,N
455             rho=Qp(1,j);u=Qp(2,j);p=Qp(3,j);T=Qp(4,j)
456             write(unitno,'(5(ES22.13,1X))') xc(j),rho,u,p,T
457         end do
458
459         write(unitno,*)
460     end subroutine
461
462     subroutine write_exact_solution(unitno)
463         integer,intent(in)::unitno
464         integer::j
465         real(dp)::rho,u,p,T
466
467         do j=1,N
468             rho=Qp_exact(1,j);u=Qp_exact(2,j);p=Qp_exact(3,j);T=Qp_exact(4,j)
469             write(unitno,'(5(ES22.13,1X))') xc(j),rho,u,p,T
470         end do
471
472         write(unitno,*)
473     end subroutine
474
475     subroutine write_residual(unit,t,res)
476         integer,intent(in) :: unit
477         real(dp),intent(in) :: t,res(:,:)
478         real(dp) :: max_res(3)
479         integer :: k
480
481         do k = 1, 3
```

```
            max_res(k) = maxval(abs(res(k,:)))
        end do

        write(unit,'(4ES20.10)') t, max_res(1), max_res(2), max_res(3)
    end subroutine

    subroutine get_riemann_states(which, QpL, QpR)
        integer, intent(in) :: which
        real(dp), intent(out) :: QpL(4), QpR(4)

        call ic(which)

        QpL = Qp(:,1)
        QpR = Qp(:,N)
    end subroutine get_riemann_states

end module sub
```

B Python Post-Processing

```

from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt
import gc

# Configuration
DATA = Path("../code/")
OUT = Path("plots")

SELECTED_IC = 1

N_FLUX = 5
N_TIME = 2

if SELECTED_IC not in (1, 2, 3):
    raise ValueError("SELECTED_IC must be 1, 2, or 3")

OUTPUT_INTERVAL = 0.0002
TEND = 0.002

```

```
EXACT_TIMES = np.arange(0.0, TEND + OUTPUT_INTERVAL/2, OUTPUT_INTERVAL)

VARIABLES = {
    "rho": {
        "column": 1,
        "label": r"$\rho$",
        "title": "Density",
    },
    "u": {
        "column": 2,
        "label": r"$u$",
        "title": "Velocity",
    },
    "p": {
        "column": 3,
        "label": r"$p$",
        "title": "Pressure",
    },
    "T": {
        "column": 4,
        "label": r"$T$",
        "title": "Temperature",
    },
}

FLUX_NAMES = {
    1: "Steger-Warming",
    2: "Van Leer (Local)",
    3: "Van Leer (Common)",
    4: "AUSM",
    5: "Roe",
}

TIME_NAMES = {
    1: "Forward Euler",
    2: "Runge-Kutta 4",
}

# Plot style
plt.rcParams.update({
    "text.usetex": True,
```

```
    "font.family": "serif",
    "font.serif": ["Computer Modern Roman"],
    "axes.labelsize": 18,
    "axes.titlesize": 16,
    "xtick.labelsize": 13,
    "ytick.labelsize": 13,
    "legend.fontsize": 11,
    "axes.linewidth": 1.2,
    "xtick.direction": "in",
    "ytick.direction": "in",
    "xtick.top": False,
    "ytick.right": False,
    "xtick.major.size": 7,
    "ytick.major.size": 7,
    "xtick.minor.size": 4,
    "ytick.minor.size": 4,
})

# Directory structure
OUT.mkdir(parents=True, exist_ok=True)

# Formatting
def format_axes(ax, ylabel):

    ax.set_xlabel(r"$x$")
    ax.set_ylabel(ylabel)

    ax.grid(False)

    ax.tick_params(which="major",direction="in",length=7,width=1,pad=7)
    ax.tick_params(which="minor",direction="in",length=4,width=0.8)

    for spine in ax.spines.values():
        spine.set_linewidth(1.2)
        spine.set_color("black")

# Read blank-line-separated solution blocks
def read_blocks(filename, ncols=5):

    blocks = []
    block = []
```

```
104
105     with open(filename, "r") as f:
106
107         for line in f:
108
109             line = line.strip()
110
111             if not line:
112                 if block:
113                     blocks.append(np.asarray(block, dtype=float))
114                     block = []
115                 continue
116
117             if line.startswith("#"):
118                 continue
119
120             values = line.split()
121
122             if len(values) != ncols:
123                 continue
124
125             block.append([float(v) for v in values])
126
127     if block:
128         blocks.append(np.asarray(block, dtype=float))
129
130     if not blocks:
131         raise ValueError(f"No data blocks found in {filename}")
132
133     return blocks
134
135 # Read numerical and exact solutions
136 def load_case(ic, flux, time_integrator):
137
138     numerical_file = (DATA / f"results-IC{ic}-F{flux}-T{time_integrator}.dat")
139     exact_file = DATA / f"exact-IC{ic}.dat"
140
141     if not numerical_file.exists():
142         raise FileNotFoundError(numerical_file)
143
144     if not exact_file.exists():
```

```
145         raise FileNotFoundError(exact_file)
146
147     numerical = read_blocks(numerical_file)
148     exact = read_blocks(exact_file)
149
150     numerical_times = np.arange(len(numerical)) * OUTPUT_INTERVAL
151
152     if len(exact) != len(EXACT_TIMES):
153         raise ValueError(
154             f"{exact_file}: expected {len(EXACT_TIMES)} "
155             f"exact blocks, found {len(exact)}"
156         )
157
158     if len(numerical) != len(numerical_times):
159         raise ValueError("Unexpected numerical block count")
160
161     return numerical, exact, numerical_times
162
163 # Plot numerical vs exact solution
164 def plot_solution(ic, flux, time_integrator, time, numerical, exact):
165
166     # Output directory
167     directory = (
168         OUT /
169         f"IC{ic}" /
170         f"F{flux}" /
171         f"T{time_integrator}"
172     )
173
174     directory.mkdir(parents=True, exist_ok=True)
175
176     # Create 2x2 figure
177     fig, axes = plt.subplots(2, 2,figsize=(11, 8))
178     axes = axes.flatten()
179
180     # Subfigure labels in report order
181     panel_labels = ["(a)", "(b)", "(c)", "(d)"]
182
183     for ax, (variable, info), panel in zip(axes, VARIABLES.items(), panel_labels):
184
185         col = info["column"]
```

```
186
187         # Numerical solution
188         ax.plot(
189             numerical[:, 0],
190             numerical[:, col],
191             color="blue",
192             linewidth=1.8,
193             label="Numerical",
194         )
195
196         # Exact solution
197         ax.plot(
198             exact[:, 0],
199             exact[:, col],
200             color="black",
201             linestyle="--",
202             linewidth=1.6,
203             label="Exact",
204         )
205
206         format_axes(ax, info["label"])
207         ax.set_title("")
208
209         ax.legend(
210             loc="best",
211             frameon=True,
212             fancybox=False,
213             framealpha=1,
214             edgecolor="black",
215         )
216
217         # Panel label below each subplot
218         ax.text(
219             0.5,
220             -0.28,
221             panel,
222             transform=ax.transAxes,
223             ha="center",
224             va="top",
225             fontsize=15,
226         )
```

```
227
228     fig.tight_layout()
229
230     fig.subplots_adjust(
231         hspace=0.55,
232         wspace=0.30,
233         bottom=0.12,
234     )
235
236     filename = directory / f"t{time:.4f}.png"
237
238     fig.savefig(filename,dpi=600,bbox_inches="tight")
239
240     plt.close(fig)
241     del fig, axes
242
243 # Main post-processing loop
244 ic = SELECTED_IC
245
246 print(f"\nProcessing IC {ic}")
247
248 for flux in range(1, N_FLUX + 1):
249
250     for time_integrator in range(1, N_TIME + 1):
251
252         print(
253             f" Flux: {flux} ({FLUX_NAMES[flux]}) | "
254             f"Time integrator: {time_integrator} "
255             f"({TIME_NAMES[time_integrator]})"
256         )
257
258         try:
259             numerical, exact, times = load_case(
260                 ic,
261                 flux,
262                 time_integrator,
263             )
264
265             for time, data in zip(times, numerical):
266
267                 # Find corresponding exact snapshot
```

```
                exact_index = np.argmin(
                    np.abs(np.asarray(EXACT_TIMES) - time)
                )

                if not np.isclose(
                    EXACT_TIMES[exact_index],
                    time,
                    atol=1e-9,
                ):
                    print(f" Skipping t={time:.4f}: " "no matching exact snapshot")
                    continue

                plot_solution(
                    ic,
                    flux,
                    time_integrator,
                    time,
                    data,
                    exact[exact_index],
                )

            del numerical, exact, times
            gc.collect()

        except (FileNotFoundError, ValueError) as error:

            print(f" WARNING: {error}")
            continue

print("\nPost-processing complete.")
```